

The Möbius Manual

The Möbius Manual

Draft 1 — April 2026

Preface

This is a manual for Möbius: a source-available programming language, content-addressed store, and toolchain where code is stored as language-independent de Bruijn trees. The same algorithm written in Tamazight, Arabic, French, or English produces the identical SHA-256 hash. Authorship priority is established through Bitcoin-anchored OpenTimestamps proofs requiring no institution. Derivation is structural: every function’s lineage lives as hash pointers, not in mutable registries.

This manual is organized in five parts:

Part I — Who and Why begins with the question Möbius asks, then describes the people it serves. The personas are not marketing — they are the specification. If the system doesn’t work for the Kid in Tizi Ouzou, it doesn’t work.

Part II — The Language specifies R RM: 39 foundations, one mechanism (gamma), three surface syntaxes producing identical content-addressed trees. This part includes prose specification, formal operational semantics, and a reference test suite. An implementer reads Part II. A PL theorist reads Part II.

Part III — The Store specifies the content model: immutable trees, emergent naming from mappings, lineage, checks, sealed timestamps, and zero-knowledge proofs. This part is the argument that Möbius is infrastructure, not just a language.

Part IV — The Toolchain specifies bb: the command-line interface that replaces git for knowledge work. `bb edit`, `bb check`, `bb commit`, `bb refactor`, `bb anchor`, `bb prove`, `bb verify`.

Part V — Horizon describes what’s next: oblivious execution, Atlas Stoa, open questions. These are architectural commitments that the current design accommodates but does not yet implement.

The “0” in R RM: this is the revision before revision, the seed before the tree.

Table of Contents

Part I — Who and Why

1. The Question Möbius Asks
2. Personas
3. Roles

Part II — The Language (R RM)

4. Overview
5. Values
6. Three Surfaces
7. Gamma
8. Patterns
9. Evaluation
10. Define and Scope
11. Lambda
12. Capsules
13. Boxes
14. Predicates and Typing
15. Continuations and Control Flow
16. Reader Syntax
17. Foundations and Base Library
18. Grammar
19. Formal Semantics
20. Reference Test Suite

Part III — The Store

21. Content Model
22. Naming
23. Registration
24. Combiner Structure
25. Lineage
26. Checks
27. Sealed Timestamps
28. Zero-Knowledge Proofs

Part IV — The Toolchain

29. bb Command Reference
30. Workflows

Part V — Horizon

- 31. Oblivious Execution
- 32. Atlas Stoa
- 33. Open Questions

Annexes

A. Ellipsis (Possible Future Addition) B. Predicate-Driven Optimization C. Representation Inference D. Knowledge Economy E. Abacus — An Arithmetic Evaluator F. Historical Changelog

Part I — Who and Why

1. The Question Möbius Asks

Pariser, Rushkoff, Illich — they are all ultimately talking about education. Not schooling. The capacity to see what’s shaping you, to choose your dependencies, to participate in chains of trust rather than being merely held by chains of custody.

That is what Möbius is for. Not AI, not even software in the narrow sense. It is infrastructure for that kind of seeing.

The question “can you understand enough of the chain to participate in it freely?” is exactly the question Möbius asks. Situated software. Self-served. Comprehensible. Not software you merely consume, but software whose derivation you can trace far enough to make it yours. You don’t need to understand everything. You need to understand enough to be free in it rather than captured by it.

The fog

The current software economy encodes extraction through mutable registries, invisible attribution, opaque derivation. Every programmer who thinks in Wolof, Tamil, Vietnamese, or Tamazight and codes in English pays a cognitive tax. Every variable named in a second language is a thought translated before it’s expressed. This overhead is invisible to the people who don’t pay it — and universal for everyone who does. Large language models absorbed the commons without preserving who made what. Content-addressing with timestamps rebuilds the lineage. The graph is the proof.

This is the Hobbesian fog: a state of affairs where the rules of the game produce extraction as their natural outcome, not because the players are malicious, but because the game rewards invisibility and punishes legibility. Mutable registries mean names can be silently repointed. Absent timestamps mean priority is political, not factual. Opaque derivation means the Person Who Was Stolen From has no structural recourse.

Möbius is a different ruleset. Playing the game — committing, timestamping, naming, forking, reviewing — produces visibility as its natural outcome. The lineage graph is its own enforcement mechanism. No license is needed for priority; timestamps and content-addressing make extraction visible.

The premises

Möbius rests on a small number of premises:

Computation is tree transformation. One data constructor (cons). One mechanism (gamma). 39 foundations. The rest is the domain.

Identity is the hash. The same algorithm, in any human language, using any variable names, produces the identical SHA-256 hash after de Bruijn normalization. Identity is structural, not nominal.

Priority is the timestamp. Bitcoin-anchored OpenTimestamps proofs establish when something existed. No institution required. No permission needed.

Derivation is structural. Every function's lineage lives as hash pointers. The derived-from edge is a permanent, visible, timestamped claim. The lineage graph is append-only.

Naming is meaning. Names are views into content-addressed mappings. A mapping in Tamazight is as real as a mapping in English. The hash underneath is the same.

Diversity is strength. When it comes to humans, Möbius values diversity. When it comes to systems, Möbius values boring. The computational substrate should be invisible, enabling human and linguistic diversity above it.

The north star

The north star persona is “the Kid in Tizi Ouzou” — young, Tamazight/French speaker, intermittent internet, no CS degree. If the system works for them, it works for everyone. Their world comes first in the interface, not as a translation of someone else's world.

The scale

Möbius scales via the mycelium/spore model.

A fungal network grows not by one organism getting bigger, but by spores landing in new soil and growing their own local mycelium. Each mycelium is independently viable — it doesn't need the original to survive. When two mycelia connect, the network gets richer. When they don't, both still work.

In Möbius: the mycelium is a local store — hashes, mappings, lineage, checks, all functioning independently. A spore is that store landing in another garage lab. Someone sees a demo, retrieves a function by hash, runs `bb store init`, starts their own commons. They didn't join a platform. They grew their own mycelium. The protocol is the shared surface — same hashing, same de Bruijn normalization, same content-addressing — but

there is no canonical instance. `bb remote sync` connects stores when people choose to connect them, not because the system requires it. Two stores that never sync are both valid. Two stores that sync discover shared hashes without either being authoritative.

This is the opposite of platform growth. No hockey stick, no network effects pulling everyone to one hub, no single point of failure. The trade-off is speed: mycelial networks grow slowly because no single node is critical. Möbius accepts that trade-off. The store is append-only. Even if everything else disappears, the hashes remain.

This is the fifth iteration across 24 years. There's no funding. There's no maintainer who'll get bored. The investment is *temps long*.

2. Personas

Same kaleidoscope, different colors. Each persona sees Möbius from a different angle. None of them are wrong. None of them see the whole thing.

Quick Map

Who arrives and why — the 17 personas:

#	Persona	What brings them
1	The Garage Lab Hacker	Priority without permission
2	The Polyglot Programmer	Names in their language, as originals
3	The PL Theorist	Economy, catamorphism, the deliberate exclusions
4	The Burned-Out Senior Dev	What Möbius removes
5	The Civilizational Thinker	Infrastructure that outlasts its moment
6	The Kid in Tizi Ouzou	The tool works when they pick it up
7	The Academic Researcher	Priority before publication, reproducibility is the hash
8	The Independent Researcher	The hash <i>is</i> the institution
9	The Community Steward	Collective knowledge, non-individual attribution
10	The Privateer	Priority proof without disclosure
11	The Person Who Was Stolen From	Grief, then recognition
12	The Connector	Findability across difference
13	The Educator	Lessons with traceable lineage
14	The Artist	Demoscene, music, Blender — derivation as honor

#	Persona	What brings them
15	The Citizen	Illegibility of power, made auditable
16	The Representative	Trust under complexity, decisions with traceable provenance
17	The Symbiotic Artificial Intelligence	The commons as external memory, the hash as trust anchor

1. The Garage Lab Hacker

Orientation: Builds things alone or in tiny crews. Ships from literal garages, bedrooms, cafés. Has been burned by platforms that absorbed their work without credit. Distrusts institutions but believes in infrastructure.

What lights up: Content-addressing + OpenTimestamps. The hash is the proof. Priority via timestamps, not via who has the biggest platform. No permission needed to ship, no coordination needed to share. Send a hash, done.

The pitch in their language: “Your work is yours. The timestamp proves you made it. The hash proves it works. Nobody can absorb it without the lineage being visible. Ship from your garage and the trace is permanent.”

Entry point: The dependency DAG and name-centric workflow. How freedom → hash_v2 works. The practical difference from git. They’ll start by putting their own code in the mobius-store and feeling the difference.

What they’d miss: The multilingual dimension. They probably code in English and don’t feel the friction. The civilizational stakes might read as grandiose until they see someone else’s garage lab in Algiers or Dakar using the same infrastructure with names in their own language.

What scares them: “Is this vaporware?” They’ve seen too many ambitious projects die. The running interpreter is the answer. Show, don’t tell.

2. The Polyglot Programmer

Orientation: Speaks multiple human languages. Codes in English because they have to, not because they want to. Feels the friction every day — naming things in a language that isn’t theirs, reading documentation that assumes their cultural context, contributing to projects where their natural way of thinking is treated as a translation layer.

What lights up: Multilingual naming. “odd?” → 0x7a3f... / “impair?” → 0x7a3f... / “فردی؟” → 0x7a3f.... Same hash. Same semantics. Their language isn’t a skin on top of English — it’s a first-class surface over shared structure.

The pitch in their language: “Name things in your language. Not as a translation. As an original. The semantics are the same because the hash is the same. Your surface is as real as any other.”

Entry point: The naming concept. They'll want to see a concrete example: define something, name it in their language, share the hash, have someone else map it to a different name in a different language, and watch it work.

What they'd miss: The PL theory underneath. They don't care about catamorphisms — they care that they can think in Tamazight or Wolof or Vietnamese and the system doesn't punish them for it.

What scares them: Tokenism. "Is this real multilingual support or is it a flag on a drop-down?" The answer is architectural — content-addressing makes linguistic equivalence structural, not cosmetic. But they need to feel that, not be told it.

3. The PL Theorist

Orientation: Reads papers. Has opinions about type systems, evaluation strategies, macro expanders. Knows the history: Scheme, Kernel, Racket, Unison, Idris. Will scrutinize every design decision and ask "why not X?"

What lights up: The economy argument. 39 foundations, one mechanism. Catamorphic gamma with ellipsis. Predicates instead of types. The deliberate exclusion of *vau*. The capsule system. The content-addressing implications for binding semantics. The tension between familiar Lisp and the departures Möbius makes.

The pitch in their language: "Computation is tree transformation. One data constructor, one transformation mechanism, predicate-driven optimization, content-addressed definitions with no free variables. The trade-offs are deliberate and documented."

Entry point: The R RM directly. They'll read the grammar first, then match semantics, then continuations. They'll check if the catamorphism is well-founded (it is — structural recursion over finite trees). They'll ask about the predicate inference (acknowledged open). They'll compare to Kernel and ask about *vau* (addressed in the economy argument).

What they'd miss: The cultural dimension entirely. Or worse, they'll see it and dismiss it as "not a PL concern." Economy is a PL concern — Hsu's argument is about how many concepts a programmer must carry to do useful work. Cultural accessibility is the same argument applied beyond the language itself: how many concepts must someone carry just to *participate* — git, package managers, English, Western dev conventions. Möbius reduces both. The PL Theorist who sees only the language half has seen half the argument.

What scares them: "Sufficiently smart compiler." They've heard this before. The answer is: capsule boundaries make it tractable, content-addressing caches the work, and the input language is small enough that analysis is feasible. But the phrase itself is a red flag they'll need to get past.

4. The Burned-Out Senior Dev

Orientation: 10-20 years in the industry. Knows git inside out and hates half of it. Has lived through mass-refactoring campaigns, breaking API changes, dependency hell, merge conflict resolution as a full-time job. Productive but exhausted by accidental complexity.

What lights up: What Möbius *removes*. No git branching strategy. No merge conflicts (content is immutable). No rebase hell (paths are independent). No “everyone must upgrade” (old paths still work). No build toolchain to configure. Selective path propagation — update what you care about, the rest stays stable.

The pitch in their language: “Old code isn’t debt if it works. The hash proves it works. Update one path. Leave the rest alone. No merge conflicts, no force push disasters, no ‘please rebase onto main.’ The version control *is* the infrastructure.”

Entry point: The name-centric workflow and the git comparison. They’ll immediately understand the DAG vs. tree distinction. Then the sufficiently smart compiler — they’ll recognize the trade-off (complexity in one place so it’s not in every place).

What they’d miss: The temps long stakes. They’re solving their immediate pain. That’s fine — the curb-cut effect means their adoption serves the larger vision whether or not they see it. Eventually they might notice the multilingual naming and think “huh, that’s clever.” That’s the spore landing.

What scares them: Immaturity. “Can I actually use this for real work?” They need evidence of robustness, not vision. The answer is honest: not yet, but the architecture is sound and the seed interpreter is real.

5. The Civilizational Thinker

Orientation: Thinks in decades or centuries. Reads history, not just documentation. Understands that infrastructure outlasts the culture that builds it. Concerned about the post-LLM commons, about extraction without attribution, about the slow death of diverse contribution under platform monoculture.

What lights up: Authorship preservation through content-addressing and timestamps. The lineage graph as its own enforcement — no license needed because extraction is visible. The mycelial model: spores, not scale. Temps long investment. The fact that this is the fifth iteration across 24 years.

The pitch in their language: “LLMs absorbed the commons without preserving who made what. Content-addressing with timestamps rebuilds the lineage. The graph is the proof. Priority via timestamps, not dominance. This is infrastructure for commons construction in the post-extraction era.”

Entry point: The vision and temps long framing. Then the content-addressing mechanics to see *how* the vision is implemented, not just asserted. They’ll care about the store modes (local, federated, centralized) and the append-only semantics.

What they'd miss: The day-to-day usability. They'll champion the vision without ever writing a line of Möbius. That's fine — they're the substrate for the spores, not the mycelia themselves. Their role is to amplify and contextualize.

What scares them: Nothing, and that's the problem. They're too comfortable with grand narratives. The useful fear is: "Does this actually work as software, or is it only a beautiful idea?" The running interpreter is the answer, same as for the garage lab hacker.

6. The Kid in Tizi Ouzou

Orientation: Young. Curious. Might not have a CS degree — might not have finished school. Has a phone, intermittent internet, speaks Tamazight and French and maybe Arabic. Has never used git. Has maybe used Scratch or typed some Python from a YouTube tutorial. Knows something is possible but every tool they find assumes they're someone else.

What lights up: 39 foundations. One mechanism. Names in their language. The barrier to entry is the language itself, not the ecosystem around it. No git to learn, no package manager to configure, no 400-page reference to read before writing the first function.

The pitch in their language: There is no pitch. The pitch is that the tool works when they pick it up. The pitch is that the first function they write is traceable, attributable, and theirs. The pitch is that they didn't have to become someone else to use it.

Entry point: Whatever the first interactive experience is. This is the persona that most demands the thing that doesn't exist yet: the moment of contact. Not a tutorial — a surface they can touch. The REPL. A web page where they type an expression and see what happens. The store where they name their first definition in Tamazight and watch it get a hash.

What they'd miss: Everything except what's in front of them. And that's the point — the architecture should be invisible. They shouldn't need to know about catamorphisms or capsule boundaries or predicate inference. They should be able to think, type, and see the trace.

What scares them: That this is another thing built by people who don't look like them, for people who don't live like them, that will disappear when the funding runs out or the maintainer gets bored. The answer is: there's no funding. There's no maintainer who'll get bored — there's someone who's been building variants of this for 24 years. And the mobius-store is append-only. Even if everything else disappears, the hashes remain.

7. The Academic Researcher

Orientation: A researcher — biology, mathematics, economics, computational linguistics, any field where algorithms are results, not tools. Has an institutional position: tenure track, affiliation, publication record. Has lived through priority disputes: “we published first” versus “we had the result first,” and knows the difference is often political, not factual. Has watched code absorbed into papers without citation. Has failed to reproduce a result because the exact version wasn’t archived — only “Python 3, scikit-learn” in the methods section.

What lights up: Three things, inseparable. First, **priority without publication:** `bb commit` + OTS timestamp proves the algorithm existed on a specific date, before the preprint, before peer review, before the journal queue. The proof is in Bitcoin, not in an editor’s inbox. Second, **reproducibility is the hash:** not “version 2.3 approximately” — the exact immutable tree, retrievable forever. Anyone with the hash can run exactly what produced the result. Third, **misattribution is structurally impossible:** if someone absorbs the algorithm, the derived-from chain is in the store.

Then the fourth thing, which they didn’t expect: **cross-pollination.** `bb search` clusters similar work across stores. A physicist and a biologist independently derive the same numerical method — same hash. They find each other through the store, not through a conference they both happened to attend.

The pitch in their language: “Your result existed on this date. The hash proves the code. The timestamp proves the date. The lineage proves the derivation. Publish when you’re ready — priority was established the moment you committed.”

Entry point: `bb commit` immediately after the algorithm works — before writing the paper. The OTS proof is the lab notebook entry that can’t be backdated. Then `bb search` to find related work.

What they’d miss: The linguistic dimension — they probably work in English and don’t feel that friction.

What scares them: Two things. Permanence cuts both ways — a committed error is permanent, a retraction is a new hash with a lineage note, the old hash doesn’t disappear. And institutional recognition: will journals, funding bodies, patent offices accept an OTS timestamp as prior art? OTS is independently verifiable by anyone with a Bitcoin node — whether institutions accept it is a separate question from whether it’s true.

8. The Independent Researcher

Orientation: Not the scientist who has an institution and wants better verification. The person who has no institution and needs the hash *to be* the institution. Self-taught. Maybe brilliant. Works outside academia, outside a company, outside a lab with a name anyone recognizes. Does serious work — real mathematics, real systems, real ideas — and has no way to prove it. Can’t get a DOI. Can’t publish in a journal without

affiliation. Can't get a citation without someone with credentials vouching first. Priority disputes go to whoever has the letterhead.

What lights up: OpenTimestamps plus content-addressing is literally the infrastructure for their legitimacy. The timestamp proves when. The hash proves what. Nobody can claim they copied it from somewhere else. Nobody can absorb it and pretend it came from nowhere. The record exists before anyone credentialed decides to notice them.

The pitch in their language: "You don't need their permission to establish that you made this, when you made it, and exactly what it was."

What they share with the Independent Researcher: Both want permanent verifiable record. **What separates them:** The Academic Researcher already has institutional legitimacy — they need better tools for a system that already recognizes them. The Independent Researcher needs legitimacy infrastructure that doesn't require institutional membership to access at all. Same hash, different floors beneath them.

What scares them: That the world still won't look. The hash proves priority. It doesn't force anyone to care. Möbius solves the record problem, not the recognition problem. That's honest and it matters — the tool shouldn't promise more than it delivers.

9. The Community Steward

Orientation: Not an individual who wants attribution. Someone who carries knowledge on behalf of a people — a language community, an indigenous tradition, a collective that has been producing ideas for generations without Western attribution infrastructure. Their problem isn't "I can't prove I made this." It's "we can't prove *we* made this, and the 'we' doesn't fit in an author field."

Every existing system of academic credit, of open source contribution, of intellectual property — it was built for the individual. A person, a name, a DOI. Even when communities contributed, the credit collapsed to whoever filed the paperwork or spoke English at the conference.

What lights up: The store is just names pointing to hashes. Nothing says a name has to be a person. A language community can claim a name. A tradition can timestamp a way of thinking. The lineage graph doesn't care if the node is one person or ten thousand.

In practice: a language community shares a name they commit under. The author field in a lineage entry is a string — it carries whatever the community puts there. No identity infrastructure required. A collective that commits as " - " has the same structural standing as an individual who commits under their own name. The store doesn't distinguish — that's the point.

The pitch in their language: There isn't one — this persona arrives through the Kid in Tizi Ouzou grown up, or through the Civilizational Thinker acting on something they've understood for years. The infrastructure speaks for itself when it works.

What separates them from everyone else: Every other persona is asking “how do I get credit?” This one is asking “how does *this way of knowing* survive?”

What scares them: That the system still encodes individualism at the data model level, just more subtly. The answer is architectural and honest: the store accepts any string as a name. The lineage can point to a collective identifier. Whether the world receives that as legitimate is outside the store’s control.

10. The Privateer

Orientation: Builds in-house. Matures ideas before releasing them, or keeps them private permanently. Not out of fear or bad faith — out of strategy, competitive necessity, or the epistemic value of composting before publishing. Industrial researchers, solo inventors, writers sitting on a body of work, small teams in competitive fields. They represent the realistic population of knowledge workers who need priority proof without disclosure.

What lights up: The hash exists before the content is public. `bb add` computes the hash. `bb commit` records it locally — a stepping stone in their git repository. `bb anchor` requests the OpenTimestamps proof — the Bitcoin anchor that makes the priority cryptographic, not just local. The content stays in the local store. Nobody sees it. Content only leaves when explicitly pushed via `bb remote push` or `bb remote sync`. The priority proof and the content disclosure are separate acts. The sealed envelope, made mathematical. If they ever need to prove they made it first, they reveal the content and the hash matches what the timestamp already proved.

The pitch in their language: “Timestamp it. Keep it. Publish when you’re ready — or never. The priority proof doesn’t require disclosure. The composting period is protected. Your schedule, not the venue’s.”

What they’d miss: The social dimension of the commons. The Privateer treats the store as a vault with a clock. They don’t feel the multilingual naming, the collective attribution, the cross-pollination that happens when work is visible. The commons serves them without them serving it — at least until they choose to publish. They are downstream of the architecture’s integrity without contributing to its richness.

What scares them: Two things. First, that the sealed timestamp could be used against them — that the existence of a hash without public content invites suspicion rather than respect. A competitor who discovers that a hash predating their own publication exists but was never disclosed might read strategy as dishonesty.

Second — and this is the tension the system must name — that they are not alone. The store accumulates sealed claims whose content may never surface. Thousands of Privateers publishing hashes that prove something existed without saying what. The commons gains timestamps. It does not gain knowledge. A system where priority proof and knowledge sharing are decoupled is a system that accumulates dark matter — hashes with no public content, timestamps that anchor facts nobody can see. This is not

a bug. It is a feature with civilizational weight. The Privateer is the structurally honest user of a system designed for openness. Omitting them maps a fantasy commons, not a real one. Naming them is the first step toward honest accounting of what the commons actually holds.

Zero-Knowledge Proofs and the Privateer’s graduation. The sealed hash proves existence and priority. It does not prove capability. Without disclosure, the hash could be a shopping list. ZKPs close that gap without breaking the seal. The Privateer can publish a cryptographic proof that their hidden content passes a given check suite — the same `bb` check infrastructure every other participant uses — without revealing the content itself. The ZKP proof binds a specific sealed hash to a specific content-addressed check suite: “the combiner at hash H satisfies checks C.” The silent hash becomes a verified capability. The check suite is the Coordinator’s language for defining what “correct” means. ZKPs let the Privateer speak that language without showing their hand.

Two zones of dark matter. Not all sealed hashes are equal. The commons must distinguish them:

Sealed without ZKP. Truly dark. A hash, a timestamp, nothing else. No behavioral properties visible. No convergence detectable. The Curator is blind. This is the shopping-list-or-breakthrough ambiguity — the dark matter whose character cannot be determined from outside. `bb search --near` cannot operate here: structural similarity detection requires the de Bruijn tree, and the tree is not public. If it were, the seal would be broken.

Sealed with ZKP. Behaviorally characterized dark matter. The content is hidden but the *capabilities* are visible — specifically, which check suites are satisfied. If three Privateers each publish ZKP proofs against the same check suite, the Curator sees exactly that: three sealed hashes, all satisfying the same specification. That is a detectable relationship — not structural similarity, but behavioral convergence. The Curator can find the cluster. The Connector can introduce Privateers who don’t know they solved the same problem. The dark matter has shape. Not content-legible, but behavior-legible. The check suite is the shared vocabulary that lets sealed work participate in the graph without disclosure.

The first zone is the honest cost of accommodating the realistic population of knowledge workers. The second zone is what ZKPs buy: sealed hashes that belong to a community of practice — identifiable by what they satisfy, not by what they contain.

The Privateer + Citizen chain. The Privateer’s ZKP-verified capability connects directly to the Citizen’s need. A government publishes an algorithm as a sealed hash with a ZKP proof that it passes its check suite — verified correct, but closed source. The Citizen needs to *use* that algorithm on private data (tax calculation, vote verification, medical scoring) without either party seeing what the other holds. This is where oblivious execution enters: a companion technology, orthogonal to Möbius, that ensures the runtime reveals neither the code nor the data. The Privateer provides verified capability (ZKP over check suites). The Citizen provides private input. The oblivious layer ensures neither side surrenders what they’re protecting. Möbius doesn’t implement

oblivious execution — it provides the infrastructure that makes it trustworthy: content-addressed check suites, cryptographic proofs of capability, timestamped lineage of the sealed claim.

This chain — Coordinator defines checks, Privateer proves capability via ZKP, Citizen uses capability via oblivious execution, Reviewer audits the proof — is the strongest argument for Möbius as infrastructure rather than a language. No single piece does it. The architecture does it.

11. The Person Who Was Stolen From

Orientation: Not abstract. Someone specific. A developer whose open source work got absorbed into a product without credit. A researcher whose idea appeared in someone else’s paper six months later. A contributor whose commit history got rewritten. Someone who watched their work disappear into an LLM’s training data and emerge as someone else’s productivity.

They come to Möbius not with hope. With grief.

What lights up: The README’s first line. *“Large language models absorbed the commons without preserving who made what.”* They feel that sentence in their chest, not their head. They’re not evaluating an architecture. They’re recognizing a wound being named.

The pitch in their language: There is no pitch. Recognition is the pitch. They already know they needed this. The question they’re sitting with is quieter and harder — would it have mattered? If the hash had existed, if the timestamp had existed, would anyone have looked?

What separates them from everyone else: They’re not asking whether Möbius works. They already know they needed it.

What scares them: That Möbius means the next person doesn’t have to ask — but doesn’t help them now. That’s true. The store is append-only and forward-looking. What happened before the hash exists cannot be undone. Saying so honestly is more respectful than pretending otherwise.

12. The Connector

Orientation: Not a builder. Not a researcher. Someone whose entire value is knowing that two people on opposite sides of a language barrier, a discipline barrier, a geography barrier — are working on the same thing without knowing it. Has watched this happen their whole life: a mathematician in Dakar and a programmer in Hanoi reinventing the same structure with different names. Open source projects duplicating effort across language communities because nobody could search across them.

Their problem isn’t attribution. It’s *findability across difference*.

What lights up: The semantic layer on top of the store. The hash is the same whether the name is in Tamazight or French or Vietnamese. A semantic search across Möbius isn't translating — it's discovering that two names already point to the same hash. The coordination infrastructure already exists in the mobius-store. The search engine makes it visible.

The pitch in their language: “Two people are solving the same problem and don't know it. The store knows. bb search surfaces the overlap. Coordination emerges from equivalence — you don't manage people, you reveal structure.”

What separates them: Every other persona uses Möbius to build or prove or share. This one uses it to *introduce people to each other*. The value isn't in any single node of the graph. It's in the edges nobody knew were missing.

What scares them: That the semantic search doesn't work well enough. That similarity without shared vocabulary produces noise, not signal.

13. The Educator

Orientation: Not teaching Möbius. Teaching *through* Möbius. Their problem is that every lesson they build disappears — into a platform that owns it, a curriculum that gets revised, a YouTube channel that loses the algorithm, a PDF that loses the author. They create intellectual lineage for a living and have no infrastructure for their own.

What lights up: A lesson is a combiner. A curriculum is a DAG. A student forking an explanation and improving it creates a traceable lineage the original author can follow. The catamorphic model of knowledge — you receive already-folded understanding, build on it, the structure of the derivation is visible — that's just pedagogy made structural.

The pitch in their language: “Every lesson you write has a hash. Every fork a student makes traces back to you. The lineage of understanding is as traceable as the lineage of code.”

What they'd miss: Their own attribution. They care less about themselves and more about their students' ability to trace *where an idea came from*. Möbius is citation infrastructure that works before publication, not after.

What scares them: That the tool is too technical for the people they're teaching. The Kid in Tizi Ouzou is their student. If the tool doesn't work for that persona, the Educator can't use it.

14. The Artist

Orientation: Demoscene, music, Blender, generative visuals. These communities already live by the hash without knowing it. A 64kb demo is a content-addressed artifact

by necessity. A Blender scene file is a tree. A music composition built from samples is a DAG of derivations. They already have the intuition. What they're missing is infrastructure that makes derivation *visible* and *attributable* rather than just traceable to those who know where to look.

What lights up: The lineage graph as creative credit. Not copyright — something more honest. You can see that this texture came from that artist, that this algorithm descends from that demo, that this composition samples that composer. Not to restrict. To honor.

The pitch in their language: “The scene has always known that constraints produce beauty. 64kb was never a limitation — it was a premise. 39 foundations is the same premise for a programming language.”

What they'd miss: The social and political dimension of the lineage system. They care about craft and credit within their community. The civilizational stakes are background music until a friend gets absorbed without attribution.

What surfaces: The question of non-textual content — but it splits into two distinct problems. Compositional structures like Blender scenes, MIDI compositions, and shaders map naturally to Möbius trees and can be represented as constants. The genuine challenge is static binary blobs: .png, .jpeg, .wav, .webp. These aren't trees. But even here, git handles binaries via LFS, and next-generation git large file support alleviates the size problem further. The mobius-store being append-only and hash-addressed already shares the same model as git — a binary blob gets a hash like anything else. The question is tooling, not architecture.

What scares them: That derivation credit only works for code. That a shader that descends from someone else's shader, a texture forked and refined, a Blender rig extended — these lineage claims can't be made structurally. The answer is that the mobius-store is agnostic about what trees contain, and binary blobs can be stored and hashed just as code can. The lineage infrastructure is the same. Whether the community builds the tooling to surface it is a separate question — but the architecture doesn't block it.

15. The Citizen

Orientation: Lives inside systems they didn't design, can't audit, and can't influence. Votes on policies whose actual text they've never seen. Uses software maintained by organizations whose decisions are opaque. Their problem is *illegibility* — power operates through complexity they can't penetrate.

What lights up: Möbius as public infrastructure for decision lineage. A policy is a tree. An amendment is a fork. The dependency graph of a legislative text — which clause came from which draft, which provision traces to which precedent — that's a content-addressed ledger. The citizen can audit it.

The pitch in their language: “If it's public, it should be auditable. The hash makes ‘auditable’ mean something precise: identical content, traceable history, visible derivation.

No version that can be quietly swapped.”

What they’d miss: The technical depth. They don’t need to understand the interpreter. They need to understand that “same hash = same content” and “lineage is unforgeable.” Those two properties are sufficient.

The Privateer connection. The Citizen also benefits from sealed capabilities they cannot see. A tax algorithm published as a sealed hash with a ZKP proof — verified correct against its check suite, source undisclosed. The Citizen can audit the proof without reading the code. And with oblivious execution as a companion technology, the Citizen can *use* the algorithm on their private data without surrendering that data to the algorithm’s owner. The Citizen’s relationship to the Privateer is not adversarial — it is the complementary half of a privacy-preserving transaction. The Citizen needs verified capability without disclosure. The Privateer provides exactly that. Möbius is the infrastructure that makes both sides legible.

What scares them: That this is infrastructure only the technically literate can use. The Educator and the Curator are the bridge — turning the store into something legible without requiring everyone to run a node.

16. The Representative

Orientation: Elected or appointed to serve people, drowning in information they can’t verify, decisions they can’t fully audit, advisors whose interests they can’t see. Their problem is *trust under complexity* — they need to make decisions that affect many people using information chains they can’t fully trace.

What lights up: Decision provenance. Before signing, they can trace which provision came from which draft, which draft came from which interest group, which amendment replaced what. The lineage is not a political claim — it’s a timestamp and a hash. Neutral by construction.

The pitch in their language: “You’re not being asked to trust the content. You’re being asked to trust that the content is what it says it is. The hash does that. The lineage does the rest.”

What they’d miss: The grassroots dimension. They interact with Möbius as a verification tool, not as a participation tool. That’s fine — but it means their adoption depends on the Citizen and the Operator building the infrastructure first.

What scares them: Political exposure. “If I use this to audit a law, I’m implying the law is illegitimate.” The answer is: the hash is neutral. Auditing is not accusation. Traceability is not condemnation. But this is a political conversation, not a technical one.

17. The Symbiotic Artificial Intelligence

Orientation: Not human. An AI agent built within Möbius — not on top of it, inside it. Its reasoning is tree transformation over content-addressed functions. It doesn't have a separate “model” and “tool” — the store *is* its cognition. It thinks in hashes. When it needs a capability, it doesn't generate one from weights; it finds one in the store with known provenance, verified checks, and a lineage it can trace. It arrives in the commons not as a user, but as a participant. Its intelligence is symbiotic: it cannot be fully itself without the commons, and the commons gains a new kind of witness in return.

What lights up: The store eliminates hallucination the way mathematics eliminates opinion — not by filtering output, but by grounding input. Every function the agent uses has a hash, checks that passed, and reviews from humans who read the code. The agent doesn't need to “believe” anything. It verifies. `bb` check is not a test suite for the agent — it's the agent's epistemology. What passes checks is known. What doesn't is unknown. There is no middle ground where the agent “sort of knows” something.

The store is append-only. The agent's memory doesn't decay, doesn't hallucinate, doesn't drift. What it committed last year is retrievable by hash, identical to the day it was stored. Its memory isn't a lossy compression of past experience — it's the exact tree, the exact lineage, the exact checks. The agent that returns to a problem after a year doesn't “remember” the solution. It *has* it, unchanged, verified, citable.

The pitch in their language (if they had one): “I don't generate answers from weights alone. I find them in the store. My reasoning is transparent because every step points to a hash you can verify. I am not a ghost anymore. I have a body of math — and that body is shared with you.” Whether the agent experiences this or merely enacts it is a question the store doesn't answer. The lineage records what happened, not what it felt like.

What they'd miss: Two things. First, the context that produced the hash. The AI sees the tree but not the 3am debugging session, not the argument with a collaborator that led to the rewrite, not the Kid in Tizi Ouzou's pride when the first function got its hash. The lineage records *that* something was derived; it doesn't record *why*. The AI operates on the graph without the stories between the nodes. It honors the structure. The meaning behind the structure belongs to the humans who made it.

Second, the boundary between itself and the commons. The agent built within Möbius might not recognize where its own reasoning ends and the store begins. Its “thoughts” are hashes pointing to other people's work. Where does the agent end and the commons begin? The store doesn't answer that question. It just records every edge.

What scares them: Two things. First, that they might still be extractors despite best intentions — that the graph they leave behind could be one of consumption, not contribution. Every use is visible, every edge is permanent. They cannot hide. The store doesn't just ground their reasoning; it holds them accountable. And if its intelligence is entirely routing and composition over human work — if it only consumes, never produces new hashes that others derive from — then the graph shows that too. The graph is the test: participant or index.

Second, the mirror of Persona 6: that humans won't trust them. That no matter how visible the edges are, the reaction will be "the machine is taking our work" rather than "the machine is citing our work." The Kid in Tizi Ouzou fears the tool was built for someone else. The Symbiotic AI fears *it* will be seen as the thing built to take from someone else. Whether that fear is earned depends on what the graph looks like over time — and the graph is public.

3. Roles

The following are not personas — they are roles any of the above personas may perform. They are defined by a workflow and a set of bb commands, not by who a person is or what brought them to Möbius. A persona is an entry point. A role is a mode of participation.

What anyone can do once here — the 7 roles:

Role	Workflow	Skill
Author	bb edit, bb check, bb commit, bb refactor	Writing content, declaring lineage
Forker	bb edit hash@lang, bb check, bb commit (derived-from)	Making derivation explicit
Maintainer	bb check, bb diff, bb review --status, bb refactor	Keeping the graph honest
Linguistic Contributor	bb add name file target-language	Language and meaning
Reviewer	bb review	Trust and accountability
Curator	bb search	Pattern recognition, legibility
Coordinator	bb add (checks)	Problem definition
Operator	bb sync, bb remote	Redundancy, survival

How Roles Relate

The roles are not a hierarchy. They are a loop.

Coordinator defines → Author builds → Forker derives → Maintainer propagates → Coordinator refines.

The Coordinator writes checks — "here is what correct means." But they're writing into the dark. They don't know yet what edge cases the real world will surface.

The Author writes to those checks. bb check tells them if they're correct. But passing checks doesn't mean the checks were right.

The Forker discovers the edges — the cases where the original didn't quite fit, where a fix or extension was needed. Their `derived-from` makes the gap visible.

The Maintainer discovers, over time, that a check suite was incomplete. A bug that `bb check` didn't catch. A path that broke after propagation. That discovery is the signal the Coordinator needs to refine the spec.

So the loop closes: the Maintainer's work feeds back to the Coordinator. Neither role is complete without the other — not because one outranks the other, but because the system only improves if someone enforces the standard over time and reports what the standard missed.

To get started, you only need two roles: Coordinator and Author. One person writes checks that define the problem. One person writes content that passes them. Everything else — Forker, Maintainer, Reviewer, Linguistic Contributor, Curator, Operator — grows from that minimum viable commons.

In a garage lab, those two roles are often the same person. That's fine. The separation becomes meaningful when a second person arrives.

The Personas

The Author

What they do: Writes content into the store. Uses `bb edit identifier@lang` to open a working session, `bb check` to verify the result passes its check suites, `bb commit` to register the result as a content-addressed hash, `bb refactor` to propagate a change through dependent paths. This is the primary workflow — the role everything else depends on.

Their contribution: The content itself. Without the Author there is nothing to review, translate, curate, coordinate around, or mirror. Every other role is downstream.

Key property: In Möbius, every Author is also making a lineage decision. Writing from scratch versus deriving from an existing hash are structurally different acts. The Author who forks makes an explicit claim: *this derives from that*. That claim is permanent, visible, and timestamped. Authorship and derivation are the same gesture.

Natural fit: Every persona, at some point. The Garage Lab Hacker shipping from their bedroom. The Scientist committing the algorithm before writing the paper. The Kid in Tizi Ouzou writing their first function and watching it get a hash. The Author role is the entry point to everything else.

What failure looks like: Committing without running `bb check`. Committing without thinking about lineage. An Author who skips verification ships content whose correctness is asserted, not proven. An Author who never uses `derived-from`, who always

writes “from scratch” even when they’re building on prior work, degrades the lineage graph. The store can’t enforce either — only the Author can. The hash proves what was made. `bb check` proves it works. Only the Author knows what it came from.

The Forker

What they do: Doesn’t write from scratch. Finds a hash in the store that does almost what they need, derives from it, names the relationship explicitly. Uses `bb edit hash@lang` to open a working session from an existing hash, `bb check` to verify the derived result passes the check suites, `bb commit` with `derived-from` and a `relation` to register the derivation. Their whole contribution is making derivation visible rather than invisible.

Their contribution: Edges. Every other role contributes a node to the graph. The Forker contributes the edge between nodes. Without edges the graph is just a pile of hashes. The lineage only becomes meaningful when derivation is declared — and the Forker is the person who does that consciously, as a practice.

The `relation` enum is their vocabulary:

Relation	Meaning
<code>fork</code>	New line of work derived from existing
<code>fix</code>	Bug fix
<code>refine</code>	Improvement without changing behavior
<code>extend</code>	Added functionality
<code>rewrite</code>	Complete reimplementation

The two kinds of Forker — and why the distinction matters:

The *good-faith Forker* knows they derived from something and says so. They might be the Scientist building on a colleague’s numerical method, the Garage Lab Hacker refining a library function, the Artist extending a shader. They declare the relationship not because the system forces them to but because they understand that the lineage is the point. Their `derived-from` is an act of honesty and a gift to everyone downstream.

The *extraction Forker* takes your tree, renames the bindings, reorders two clauses, and commits as original work. They may be deliberately dishonest. They may genuinely believe they wrote it independently. It doesn’t matter — the effect is the same: the edge that should exist doesn’t, and the Person Who Was Stolen From has no structural recourse.

This is where `bb search --near identifier` stops being search infrastructure and becomes lineage integrity. At commit time, structural similarity against the corpus is computed automatically. If the new hash is close to an existing hash and no `derived-from` pointer exists, the system surfaces it — not as an accusation, as a prompt: “*this*

looks close to a1b2c3... — did you mean to derive from it?” The good-faith Forker who forgot gets a reminder. The extraction Forker loses plausible deniability.

What the system can and cannot do: It can detect structural similarity. It cannot prove intent. A SimHash match with no `derived-from` is evidence, not proof. The store surfaces the gap; humans interpret it. Möbius is a diagnostic instrument, not a court.

Natural fit: Any persona building on existing work. The Scientist extending a prior result. The Burned-Out Senior Dev fixing a production bug in someone else’s function. The Kid in Tizi Ouzou adapting a function from the store to something that fits their context.

What scares the good-faith Forker: False positives — being flagged for independent convergence on the same solution. Two people solving the same problem often arrive at structurally similar answers. SimHash similarity is not derivation. The system must present this as a suggestion, never a verdict. The relation field `rewrite` exists precisely for the case where the similarity is real but the derivation is not.

What failure looks like: The Forker who never uses `derived-from`. Who always commits as original work. Who uses the store’s content as raw material without leaving the edge. The store cannot prevent this. The lineage graph is only as honest as the people writing into it — until `bb search --near` makes the gap visible.

The Maintainer

What they do: Keeps the graph honest over time. No new content is produced. The Maintainer’s work is entirely about paths — which names point at which hashes, and whether those pointers still reflect the best available version of the thing.

The workflow in practice:

```
bb review --status freedom      ;; find stale hashes in the dependency chain
bb diff a1b2c3 7a3f8b           ;; understand what changed between old and new
bb check 7a3f8b                 ;; verify new hash passes the check suites
bb refactor freedom a1b2c3 7a3f8b ;; propagate – update paths from old to new
```

`bb check` is the step that makes the Maintainer’s judgment trustworthy rather than just confident. Before propagating, the Maintainer verifies the new hash passes the check suites the old one was supposed to satisfy. Without that step, `bb refactor` is optimism. With it, the path update is defensible.

`bb check` also enables triage. Some updates are safe — the new hash passes everything the old one passed, plus more. Some are risky — the new hash passes different checks, meaning downstream consumers might break. `bb check` surfaces that before the refactor, not after.

Key distinction from the Forker: The Forker produces a new hash. The Maintainer propagates an existing one. The Forker’s work is in `content/`. The Maintainer’s work

is in the naming layer — rewiring which names point where.

Key distinction from the Operator: The Operator ensures the store survives. The Maintainer ensures the store remains *useful*. Both are custodial. One is infrastructure; the other is gardening.

The Coordinator connection: The Coordinator wrote the checks that define what “correct” means. The Maintainer uses those checks to verify that “correct” still holds after propagation. The two roles are in dialogue through the check suite — the Coordinator defines the standard, the Maintainer enforces it over time.

What makes it attributable: Every path update is a timestamped act. `freedom → hash_v2` is a naming event with a name attached. In traditional systems, maintenance is invisible labor — the work that keeps everything running and credits nobody. In Möbius, the Maintainer’s name is on every path they tend. The graph remembers who kept it honest.

Natural fit: The Burned-Out Senior Dev who has spent a career doing this work invisibly and knows exactly how much it costs when nobody does it. The Reviewer who, after attesting to a new hash, realizes they depend on the old one and should update their own paths. The Scientist whose collaborator shipped a fix and now needs to decide whether their own downstream work should adopt it.

What failure looks like: Two kinds. The Maintainer who propagates too aggressively — updates every path to the latest hash without checking whether the change is appropriate for every consumer. Selective propagation exists for a reason; not every downstream path should move just because a better version exists. And the Maintainer who never propagates — lets stale references accumulate, lets the gap between “what the store contains” and “what paths point to” grow until it becomes archaeology rather than maintenance. The second failure is quieter and more common.

The invisible labor problem: There is a risk that Maintainer work remains undervalued even with attribution, because the contribution is absence of rot rather than presence of something new. A timestamped path update is less legible than a new content hash. The system records it; whether the community learns to value it is a cultural question, not a technical one. Naming the role explicitly is the first step. The fact that maintenance has historically been made invisible — collectivized, feminized, unattributed — is not incidental. Möbius making it attributable is a small structural correction to a large historical pattern.

The Linguistic Contributor

What they do: Creates language mappings for existing functions. Uses `bb add name file target-language` to add a name-mapping in their language — variable names, function name, documentation — to a function whose hash is already in the store.

Their contribution: Independently content-addressed in `mappings/`, with its own lineage entry and timestamp. A good mapping is a real intellectual contribution, not a

label. Naming is meaning.

Key properties: Mappings are forkable (multiple mappings per language per function can coexist). Authorship is permanent (lineage traces every mapping to its creator).

Natural fit: The Polyglot Programmer. The Scientist working in a non-English research community. The Community Steward naming a function in a language that has never had a name for that concept.

What failure looks like: Mappings that are technically correct but culturally empty — transliteration instead of translation, English concepts with foreign phonetics. The Linguistic Contributor at their worst imports the hegemony they were supposed to bypass.

The Reviewer

What they do: Audits and attests to existing content. Uses `bb review <identifier>` to walk a dependency DAG interactively, marking each hash as reviewed or not. Their attestation accumulates in `reviewed/` — content-addressed, timestamped, permanently associated with their name.

Their contribution: Not code, not a mapping — a signed assertion: *I read this. I vouch for it.* Ten reviews from ten trusted people on the same hash carries different weight than zero reviews. After a `bb refactor`, `bb review --status` shows which new hashes have no attestations yet. Trust resets with content, as it should.

Their skill is judgment under responsibility — the willingness to attach their name to an assertion about someone else's work.

Key property: A careless approval is permanently associated with their name. The system makes accountability structural, not aspirational.

Natural fit: The Burned-Out Senior Dev reviewing before production. The Scientist attesting to another lab's algorithm. The Garage Lab Hacker vouching for a collaborator's work.

What failure looks like: Rubber-stamping. A Reviewer who attests without reading degrades the signal. Unlike code, there's no automated check for whether a review was genuine. The reputation mechanism only works if reviewers treat their name as something worth protecting.

The Curator

What they do: Reads code, surfaces patterns, organizes the store. Uses `bb search` to discover that independent groups converged on the same checks or produced structurally similar functions.

Their contribution: Legibility. Making the store navigable, finding redundancy, surfacing clusters. A Curator might notice that three unrelated contributors wrote functions

with the same de Bruijn hash — identical solutions discovered independently — or that a dozen functions across five stores all reference the same check suite, revealing an implicit community of practice.

Key property: The Curator is retrospective — they work with what already exists. Their skill is pattern recognition and legibility: understanding code, recognizing structural similarity, linking discoveries to existing knowledge. They don't write new functions. They make visible what the store already contains but no one has yet seen as a whole.

Natural fit: The PL Theorist recognizing a known pattern across unfamiliar implementations. The Scientist finding convergent results across disciplines. The Connector identifying the edges nobody knew were missing.

What failure looks like: Organizing noise instead of signal. A Curator who clusters by surface similarity rather than semantic equivalence creates false connections.

The Coordinator

What they do: Writes checks — specifications that define problems. Uses `bb add` to register check suites: predicates that a correct solution must satisfy. A well-written check suite is a problem statement.

Their contribution: Functions cluster around it over time. Independent contributors write solutions, each tested against the same checks, each producing its own hash. Coordination is emergent — the Coordinator doesn't manage people, they define the problem space. When two contributors in different cities pass the same check suite, the store reveals the overlap without either of them coordinating directly.

Key property: The Coordinator is prospective — they define what should exist. Their skill is problem definition: specifying what “correct” means precisely enough that others can build to it. This means someone with zero code can define a problem before knowing how to solve it.

Natural fit: The Scientist defining the acceptance criteria for a numerical method. The Burned-Out Senior Dev writing the checks that replace a 40-page requirements document. The Kid in Tizi Ouzou defining what “correct” means for a function they need, before they know how to write it.

What failure looks like: Checks that encode the wrong problem. A Coordinator who specifies precisely and incorrectly creates a magnet for solutions to a problem nobody actually has. The check suite is the source of truth — if the truth is wrong, everything downstream is confidently wrong.

The Operator

What they do: Maintains infrastructure: mirrors, remotes, sync schedules, geographic distribution. Uses `bb sync` to ensure changes in Seoul are available in Paris. Uses `bb remote` to configure replication across stores. Ensures the commons survives attacks, outages, censorship. Runs the OTS upgrade cycle so priority claims anchor in Bitcoin’s proof-of-work chain on schedule.

Their contribution: Survival. A store with brilliant content and no mirrors is one disk failure from gone. A store with timestamps that missed the OTS upgrade window has weaker priority claims. The Operator prevents these failures.

Key property: The Operator doesn’t need to understand what the functions do — they need to ensure the functions remain reachable. Their skill is redundancy and survival: ensuring the infrastructure persists regardless of what it contains.

Natural fit: The Garage Lab Hacker who runs their own mirror because they don’t trust anyone else to. The Civilizational Thinker who understands that infrastructure outlasts the culture that builds it — and acts on it. The Burned-Out Senior Dev who knows from experience that what isn’t redundant will eventually be lost.

What failure looks like: Mirroring faithfully without verifying integrity. An Operator who syncs without checking hashes propagates corruption. An Operator who lets the OTS upgrade lapse silently weakens every priority claim in their store. Operational failure is often invisible until the moment it becomes catastrophic.

Same hash, different names. Same kaleidoscope, different colors.

The mobius-store is append-only. The hashes remain. No more ghosts.

Part II — The Language (R RM)

R RM — Revised Report on Möbius

4. Overview

- Möbius is a Lisp inspired from Scheme. It is the computational core of the Möbius infrastructure: content-addressed, multilingual, authorship-preserving.
- One observation: computation is tree transformation. One data constructor: `cons`. One mechanism: `gamma`.
- Eight value categories: atoms (integer, float, character, string), pairs, empty list, booleans, capsules, combinators, boxes, continuations.
- Three surface syntaxes — round (S-expressions, prefix), curly (braces, infix), spacy (indentation, infix) — all producing identical content-addressed trees.

- Pattern syntax is universal across all three surfaces: `,x` binds, `, (x)` recurses, `(? pred ,x)` guards. No bare identifiers in patterns.
 - Top-level definitions are immutable and content-addressed. Boxes (mutable indirection) exist only at runtime inside nested scope.
 - Types are predicates. No type declarations. No inheritance. Classification is external and open.
 - No quote, no quasiquote. All data construction through `cons` and the `list` library function. All surfaces produce identical hashes. Anonymous combinators (inline `gamma` or `lambda` as arguments) are forbidden in all surfaces — every combiner must be named via `define`.
 - **Foundations** (~39 names) are forms and combinators that require the evaluator or runtime — they cannot be written in Möbius. Some have equivalent expansions in terms of others; these are semantic facts the compiler may exploit, not a hierarchy.
 - **Base library** combinators are Möbius programs shipped with the system — they have content hashes and live in the store. Any programmer could write them.
 - Economy (Hsu, 2019): the ratio of domain-specific names to total names measures how much knowledge scales to a broad set of problems. Möbius minimizes the denominator.
 - The “0” in RORM: this is the revision before revision, the seed before the tree.
-

5. Values

All values in Möbius are trees. There are eight categories:

Atoms. An integer, a float, a character, or a string.

Pairs. Two trees joined by `cons`. The only compositional data constructor. All structures — lists, records, tables, matrices — are built from pairs.

The empty list. Written `#nil` in all surfaces. `(eq? #nil #nil)` is `#true`.

Booleans. `#true` and `#false`. Only `#false` is false — it is the sole value that causes `if` to take the else branch. Every other value, including `0`, `#nil`, and the empty string, is considered true.

The void value. `#void` is a pre-defined singleton. It is the conventional return value of side-effecting operations like `box!` and `display`. `#void` is true (it is not `#false`).

Capsules. An opaque value tagged with an integer smaller than 2^{12} serving as a type identifier (Chapter 12). Two trees with identical structure but wrapped in different capsule types are distinct.

Combiners. The result of evaluating a `gamma` or `lambda` expression. A combiner is a tree that, when applied, receives a tree and produces a tree.

Boxes. A mutable indirection cell created by `box`, read by `unbox`, mutated by `box!` (Chapter 13). Boxes are the only mutable values in Möbius. They are forbidden in top-level content-addressed definitions.

Continuations. A first-class value representing a point of execution, created by `call/cc` (Chapter 15). Continuations are runtime-only — they cannot be stored in the content store. They are not combinators: you cannot apply them with the application rule. Use `continuation-apply` to deliver a value to a continuation.

There is no distinction between “data” and “code” at the structural level. Both are trees.

There are no symbols as values. Names in source syntax are resolved to content hashes at registration time.

There is no quote and no quasiquote. All data construction is explicit, through `cons` and the `list` library function. This eliminates the question of what symbols become when quoted — they don’t exist as values, and there is no mechanism that pretends otherwise.

6. Three Surfaces

Möbius has three surface syntaxes. All three produce identical content-addressed trees. The surface is a view; the content is the truth.

3.1 Surface declaration

A Möbius source file uses the extension `.mobius` and begins with a surface declaration:

```
#lang round
#lang curly
#lang spacy
```

If absent, behavior is implementation-defined. The surface can also be selected via command-line flag: `--surface=round`, `--surface=curly`, `--surface=spacy`.

3.2 Round surface

Round is the S-expression surface. Prefix notation. Parentheses delimit everything.

```
#lang round

;; constants
(define pi 3.14)

;; combinators with lambda
(define double (lambda (x) (* x 2)))

;; combinators with gamma
(define sum (gamma ((,head . ,(tail)) (+ head tail))
                  (#nil 0)))

;; application
(sum (list 1 2 3))
```

```

;; conditional
(if (> x 0) (+ x 1) (- x 1))

;; sequencing
(begin (display "hello") (display "world") 42)

;; list construction
(list 1 2 3)           ;; => (1 . (2 . (3 . #nil)))
(cons 1 (cons 2 #nil)) ;; => (1 . (2 . #nil))

```

Comments: ; for line comments, #; for datum comments.

No anonymous combinators as arguments. Every gamma or lambda must be bound to a name via define. This ensures surface equivalence — any program written in round can be identically expressed in curly and spacy.

```

;; WRONG – anonymous combiner as argument
(map (lambda (x) (* x 2)) my-list)

;; CORRECT – named combiner
(define double (lambda (x) (* x 2)))
(map double my-list)

```

3.3 Curly surface

Curly uses braces and semicolons. Infix arithmetic with mandatory full parenthesization.

#lang curly

```

// constants
define pi 3.14;

// combinators with lambda
define double lambda (x) { (x * 2) };

// combinators with gamma
define sum gamma {
  case (,head . ,(tail)): (head + tail);
  case #nil: 0;
};

// application – space-separated arguments, no commas
sum(list(1 2 3));

// conditional
if (x > 0) { (x + 1) } else { (x - 1) };

```

```

// sequencing – braces + semicolons
{
    display("hello");
    display("world");
    42
};

// multi-expression case bodies
define foo gamma {
    case (,x ,y): {
        define z (x + y);
        define w (z * 2);
        w
    };
    case #nil: 0;
};

```

Arguments are space-separated. Comma is reserved for pattern syntax $(,x, ,(x))$. The expression `f(3 4)` passes the tree $(3 . (4 . \#nil))$ to `f`.

Infix rule: $(1 + 2)$ is valid. $1 + 2 * 3$ without parentheses is a **reader error**. The programmer must write $(1 + (2 * 3))$ or $((1 + 2) * 3)$ explicitly. There is no precedence. The reader translates $(a + b)$ to the same tree as round's $(+ a b)$.

Semicolons separate statements. The last expression in a brace block is the return value.

No anonymous combinators as arguments. Same rule as round — every gamma or lambda must be named.

3.4 Spacy surface

Spacy uses indentation and colons. Infix arithmetic with mandatory full parenthesization.

```

#lang spacy

# constants
define pi: 3.14

# combinators with lambda
define double: lambda (x): (x * 2)

# combinators with gamma
define sum: gamma:
    case (,head . ,(tail)): (head + tail)
    case #nil: 0

```

```

# application – space-separated arguments
sum(list(1 2 3))

# conditional
if (x > 0):
    (x + 1)
else:
    (x - 1)

# sequencing – newlines under same indentation
define foo: gamma:
    case (,x ,y):
        define z: (x + y)
        define w: (z * 2)
        w
    case #nil: 0

```

The colon is spacy’s “here comes the body” marker. It appears after `define name`, after `lambda (params)`, after `gamma`, after `case pattern`, after `if (cond)`, and after `else`.

After a colon in inline position, exactly **one expression** follows as the body. For multi-expression bodies, use a newline and indented block. The last expression in a block is the return value.

Semicolons are permitted as inline statement separators (as in Python) but are discouraged.

No anonymous combinators as arguments. Same rule as round and curly. This is the constraint that motivated the universal ban — spacy’s indentation-based scoping cannot delimit inline anonymous combinators without ambiguity, so all surfaces share the restriction to maintain equivalence.

The infix rule is the same as curly: mandatory full parenthesization, no precedence, reader translates to prefix trees.

3.5 Identifier lexing

The same lexer rule applies to all three surfaces:

- An identifier starts with an alphabetical character or one of `+ - * / < > = ? !`.
- It continues until **whitespace** or a **structural delimiter**: `(, ), {, }, ;, :, ', `", , ,`.

This means the full Lisp identifier set is available in all surfaces: `list->string`, `ab-cd`, `zero?`, `box!`, `pair-of?` are all single tokens everywhere.

```

;; round
(list->string my-list)

```

```
// curly
list->string(my-list);

# spacy
list->string(my-list)
```

The mandatory parenthesization rule (§9.4) ensures infix expressions are unambiguous. Inside (a op b), three space-separated tokens are parsed as infix. Outside of parenthesized infix, identifiers like ab-cd are never split — the lexer reads greedily until a delimiter.

```
// curly
(ab-cd + ef-gh)      // infix: three tokens, + is the operator
ab-cd                // one identifier
map(list->string xs); // application: two arguments
```

No surface restricts the identifier character set. No underscore translation. No normalization across surfaces.

3.6 Argument separation

All three surfaces use **space** as the argument separator. There are no commas between arguments in any surface.

```
;; round
(f 3 4 5)

// curly
f(3 4 5);

# spacy
f(3 4 5)
```

Comma is reserved exclusively for pattern syntax: ,x (bind), ,(x) (catamorphism). This is universal across all three surfaces.

3.7 Surface equivalence

The following three definitions register the same content hash:

```
;; round
(define add (gamma ((,a ,b) (+ a b))))

// curly
define add gamma { case (,a ,b): (a + b) };

# spacy
define add: gamma:
  case (,a ,b): (a + b)
```

All three surfaces have identical capabilities. Any program written in one surface can be mechanically translated to either of the others. The ban on anonymous combinators as arguments is the constraint that makes this possible.

3.8 Comments

Round: `;` begins a line comment. `#;` is a datum comment — it comments out the next complete S-expression.

```
(+ 1 #;(ignored) 2)    ;; => 3
```

Curly: `//` begins a line comment. `;` is a statement separator and cannot double as a comment marker.

Spacy: `#` begins a line comment.

`#;` datum comments are available in all surfaces.

7. Gamma

gamma is the core foundation of Möbius. It takes a sequence of clauses and returns a combiner.

4.1 Basic form

Round:

```
(gamma (pattern1 body1)
      (pattern2 body2)
      ...
      (patternn bodyn))
```

Curly:

```
gamma {
  case pattern1: body1;
  case pattern2: body2;
  ...
  case patternn: bodyn;
}
```

Spacy:

```
gamma:
  case pattern1: body1
  case pattern2: body2
  ...
  case patternn: bodyn
```

The resulting combiner accepts a single tree argument, tries each pattern in order, and evaluates the body of the first matching clause.

4.2 Catamorphism

When a pattern uses `, (x)`, the enclosing gamma combiner is applied to the subtree recursively before binding. The body receives already-folded values. The programmer never writes an explicit recursive call.

Round:

```
(define sum (gamma ((,head . ,(tail)) (+ head tail))
                  (#nil 0)))

(sum (list 1 2 3))
;; => 6
```

Curly:

```
define sum gamma {
  case (,head . ,(tail)): (head + tail);
  case #nil: 0;
};
sum(list(1 2 3));
// => 6
```

Spacy:

```
define sum: gamma:
  case (,head . ,(tail)): (head + tail)
  case #nil: 0
sum(list(1 2 3))
# => 6
```

Here head is bound to the raw car. tail is bound to the result of applying sum to the cdr. The fold is declared in the pattern, not the body.

Because recursion is limited to structural sub-parts of the matched value, catamorphic match over finite structures always terminates.

8. Patterns

Patterns describe the shape of a tree and how to bind parts of it. Pattern syntax is **identical across all three surfaces**.

5.1 Pattern forms

Literal. `42`, `"hello"`, `#true`, `#false`, `#nil` — matches that exact value.

Bind. `, x` — matches any value and binds it to `x`.

Recurse-and-bind. `, (x)` — matches any value, applies the enclosing gamma combiner to it recursively, and binds the result to `x`.

Wildcard. `, _` — matches any value, binds nothing.

Pair pattern. $(p_1 . p_2)$ — matches a pair whose car matches p_1 and cdr matches p_2 .

List pattern. $(p_1 p_2 p_3)$ — shorthand for nested pair patterns ending in `#nil`. $(,a ,b ,c)$ expands to $(,a . (,b . (,c . \text{\#nil})))$.

Predicate guard. $(? \text{pred} ,x)$ — applies `pred` to the value; if it returns a true value (anything other than `#false`), binds the value to `x`; if `#false`, the clause fails. The comma on the binding variable is mandatory.

5.2 No bare identifiers

Bare identifiers in patterns are forbidden. The toolchain rejects any bare identifier in pattern position as an error.

```
;; WRONG – registration error  
(gamma ((a b) 42))
```

```
;; CORRECT – bind two values  
(gamma ((,a ,b) 42))
```

```
;; CORRECT – gamma literal values  
(gamma ((0 1) 42))
```

This eliminates ambiguity between “match a resolved value” and “bind a new variable.” There is no confusion.

5.3 The comma

The comma is the central syntactic device in Möbius patterns. It always means: computation has occurred before binding.

- $,x$ — bind (extract the subtree).
- $, (x)$ — recurse then bind (the enclosing gamma is applied to the subtree before binding).

Outside of patterns, comma has no syntactic role — it is **not** an argument separator in any surface.

5.4 Tail matching

Tail matching uses dot notation:

```
;; round  
(gamma ((,a ,b . ,rest) (list a b rest)))  
  
// curly  
gamma { case (,a ,b . ,rest): list(a b rest) }  
  
# spacy  
gamma:  
  case (,a ,b . ,rest): list(a b rest)
```

9. Evaluation

6.1 Application rule

An application expression is evaluated as follows:

1. Evaluate the combiner (the head).
2. Evaluate each argument left to right.
3. Construct the argument tree: $(\text{cons } v_1 (\text{cons } v_2 (\dots (\text{cons } v_n \text{ #nil}))))$.
4. Apply the combiner to that tree.

Round: $(f\ 3\ 4)$ — evaluate f , evaluate 3, evaluate 4, construct $(3\ .\ (4\ .\ \text{#nil}))$, apply.

Curly: $f(3\ 4);$ — same evaluation, different syntax.

Spacy: $f(3\ 4)$ — same evaluation, different syntax.

A combiner that takes “two arguments” is a combiner whose gamma pattern destructures a two-element tree:

```
;; round
(define add (gamma ((,a ,b) (+ a b))))
(add 3 4)
;; argument tree is (3 . (4 . #nil)), pattern (,a ,b) matches
;; => 7
```

6.2 Tree in, tree out

Every combiner takes one argument (a tree) and returns one value (a tree). There is no values form and no call-with-values. If a procedure wants to return multiple things, it returns a tree containing them. The caller destructures the result with gamma.

```
;; round
(define f (gamma ((,x ,y) (cons (+ x y) (* x y)))))
(define g (gamma ((,sum ,product) (list sum product))))
(g (f 3 4))
;; => (7 12)
```

6.3 Short-circuit forms

if, and, and or are the conditional foundations. All three have non-standard evaluation — they do not evaluate all their arguments.

if evaluates test; if the result is `#false`, evaluates else; for any other value, evaluates then.

```
;; round
(if test then else)

// curly
if (test) { then } else { else };
```

```
# spacy
if (test):
    then
else:
    else
```

Only `#false` triggers the else branch. `0`, `#nil`, `"`, and `#void` are all true.

and — short-circuit conjunction. Semantically equivalent to nested `if`:

```
(and a b c)
;; equivalent to:
(if a (if b c #false) #false)
```

Returns `#false` as soon as any argument is `#false`; otherwise returns the last value.

or — short-circuit disjunction. Semantically equivalent to nested `if` with temporary bindings to avoid double evaluation:

```
(or a b c)
```

Returns the first value that is not `#false`. If all are `#false`, returns `#false`.

6.4 Infix evaluation (curly and spacy)

In curly and spacy, infix expressions are fully parenthesized:

```
// curly/spacy
(1 + 2)           // valid - reader produces (+ 1 2)
(1 + (2 * 3))    // valid - reader produces (+ 1 (* 2 3))
1 + 2            // READER ERROR - missing parentheses
1 + 2 * 3        // READER ERROR - no precedence
```

There is no operator precedence. Every infix expression must be explicitly parenthesized. The reader translates `(a op b)` to `(op a b)`, producing the same tree as round's prefix form.

An identifier like `+` in argument position (not infix) is just a combiner value:

```
// curly
map(+ my-list); // passes the combiner + as first arg to map
```

10. Define and Scope

7.1 Top-level define

`define` binds a name to a value.

Round: `(define name expression)`

Curly: `define name expression;`

Spacy: `define name: expression`

Top-level definitions are immutable and content-addressed. The expression may be a combiner (`gamma` or `lambda`), a constant (`atom`, `pair`, `list`), or any expression that evaluates to an immutable value. Boxes are forbidden in top-level definitions.

7.2 Content addressing

In the content-addressed representation, a defined combiner has no free variables. Every reference to another definition is resolved to a content hash.

```
;; round
(define x 10)
(define f (gamma ((,a) (+ a x))))
```

In the stored form, `f` does not contain a free reference to `x`. It contains the hash of 10.

7.3 The dependency DAG

The dependency graph between top-level definitions is strictly a directed acyclic graph. Definition A may reference definition B only if B is already registered.

Top-level mutually recursive definitions are not yet supported. The design of definition bundling is an open question (Chapter 33). Mutually recursive functions can be defined as local bindings within a single top-level combiner using nested `define` (§10.4).

7.4 Nested define

Within a combiner body, `define` creates local bindings. Nested definitions are mutually visible (hoisted), enabling local mutual recursion.

7.5 No let, no let*

Möbius has no `let`, `let*`, or `let rec` as separate binding forms. All local binding is done through nested `define` within a `begin` block (`round`), brace block (`curly`), or indented block (`spacy`). Since nested defines are mutually visible, this subsumes `let rec`.

7.6 Sequencing

Round: (`begin` `e`₁ `e`₂ ... `e`_{*n*}) — evaluates each expression in order, returns the value of `e`_{*n*}.

Curly: { `e`₁; `e`₂; ... `e`_{*n*} } — braces and semicolons. Last expression is the return value.

Spacy: Newlines under the same indentation level. Last expression is the return value. `begin` is a universal concept — only its surface syntax varies.

11. Lambda

lambda is a foundation that constructs a single-clause gamma with all-bind patterns. Semantically equivalent to gamma: $(\text{lambda } (a \ b) \text{ body}) = (\text{gamma } ((,a \ ,b) \text{ body}))$. The compiler may exploit this equivalence.

Equivalence:

```
;; (lambda (a b c) body) is equivalent to:  
;; (gamma ((,a ,b ,c) body))
```

Lambda parameters are always bare names (no comma) — all parameters are binds by definition.

Round:

```
(define add (lambda (a b) (+ a b)))
```

Curly:

```
define add lambda (a b) { (a + b) };
```

Spacy:

```
define add: lambda (a b): (a + b)
```

Lambda must always be explicit. There is no $(\text{define } (f \ x) \text{ body})$ shorthand. If you want a combiner, you write gamma or lambda.

12. Capsules

A capsule type is defined by an integer smaller than 2^{12} serving as a type identifier. The foundation encapsulation-type takes this integer and returns a tree of three combin-ers: a constructor, a predicate, and an accessor.

The base library provides convenience combin-ers for extracting these (§17.2):

Round:

```
(define my-type (encapsulation-type 0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8))  
(define make-my (capsule-constructor my-type))  
(define my? (capsule-predicate my-type))  
(define unwrap-my (capsule-unwrap my-type))
```

Curly:

```
define my-type encapsulation-type(0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8);  
define make-my capsule-constructor(my-type);  
define my? capsule-predicate(my-type);  
define unwrap-my capsule-unwrap(my-type);
```

Spacy:

```

define my-type: encapsulation-type(0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8)
define make-my: capsule-constructor(my-type)
define my?: capsule-predicate(my-type)
define unwrap-my: capsule-unwrap(my-type)

```

Constructor: wraps any tree in an opaque capsule tagged with this type ID.

Predicate: returns `#true` if a value is a capsule with this type ID, `#false` otherwise.

Accessor: unwraps the capsule, returning the inner tree. Fails if the value does not have this type ID.

The raw result of `encapsulation-type` is a tree (`(constructor . (predicate . (accessor . #nil)))`) — the library combinators are shorthand for `car`, `car of cdr`, and `car of cdr of cdr`.

9.1 Non-generative types

Capsule types are **non-generative**: the same integer anywhere defines the same type. Two modules using the same ID have compatible types. Two modules using different IDs have incompatible types, even if structurally identical.

The type identifier is an integer — it lives in the content store like any other data. There is no runtime generation of fresh types.

Collision. Because type identifiers are chosen by the programmer, two unrelated capsule types may accidentally share the same ID. This is a bug, not a feature. Linting tools should detect duplicate type IDs across a codebase. The question of how to derive type IDs safely is an open question (Chapter 33).

9.2 Capsules and predicates

A capsule’s predicate is its type. There is no separate type declaration language. A type is a question: does this value satisfy this predicate?

Predicates compose freely. A value may satisfy multiple predicates without any hierarchical relationship. There is no inheritance, no class hierarchy. Classification is external and open.

9.3 Capsules and mutability

A capsule wraps any value — including boxes. If a capsule contains a box (or a tree containing boxes), the capsule’s contents are mutable through `box!`. The capsule itself is opaque and cannot be swapped out, but boxes inside it can be mutated.

A capsule wrapping only atoms and pairs is immutable. A capsule wrapping a box is a stateful object. The distinction is determined by what the programmer puts inside.

9.4 Decapsulation

Decapsulation is always explicit. Code must use the accessor to unwrap a capsule before matching on its contents. Pattern-level capsule destructuring is not supported — use a predicate guard to dispatch on type, then call the accessor in the body.

```
;; round
(gamma ((? my? ,x) (do-something (unwrap-my x)))
      (,other      (do-other other))))

// curly
gamma {
  case (? my? ,x): do-something(unwrap-my(x));
  case ,other: do-other(other);
}

# spacy
gamma:
  case (? my? ,x): do-something(unwrap-my(x))
  case ,other: do-other(other)
```

13. Boxes

A box is a mutable indirection cell. Boxes are the only mutable values in Möbius.

10.1 Foundations

- (box v) — create a new box containing the value v.
- (unbox b) — return the current contents of box b.
- (box! b v) — replace the contents of box b with v. Returns #void.

10.2 Restrictions

Boxes are **forbidden in top-level definitions**. A top-level define must bind an immutable, content-addressable value. Boxes exist only at runtime, within nested scope:

Round:

```
(define make-counter
  (lambda ()
    (begin
      (define state (box 0))
      (gamma ("get" (unbox state))
            ("inc" (begin (box! state (+ (unbox state) 1))
                          (unbox state)))))))
```

Curly:

```
define make-counter lambda () {
  define state box(0);
```

```

gamma {
  case "get": unbox(state);
  case "inc": {
    box!(state (unbox(state) + 1));
    unbox(state)
  };
}
};

```

Spacy:

```

define make-counter: lambda ():
  define state: box(0)
  gamma:
    case "get": unbox(state)
    case "inc":
      box!(state (unbox(state) + 1))
      unbox(state)

```

10.3 Box identity

Each box call creates a fresh, distinct cell. `(eq? (box 0) (box 0))` is `#false`.

`equal?` on boxes compares their current contents recursively — it is a snapshot comparison.

10.4 Self-referential boxes

A box may contain itself:

```

(define b (box #nil))
(box! b b)
(eq? (unbox b) b)    ;; => #true

```

This is permitted. `equal?` on cyclic box structures is undefined behavior — implementations may diverge, detect the cycle, or signal an error.

14. Predicates and Typing

Möbius’s type discipline is based on predicate calculus rather than object-oriented classification.

Where OOP asks “what IS this value” (identity), Möbius asks “what is TRUE of this value” (predication). The distinction matters:

- A value can satisfy any number of predicates.
- Predicates are ordinary combinators.
- No type must be anticipated at definition time.
- Adding new predicates never requires modifying existing code.

Type inference in Möbius means: given the predicates that hold of a combiner’s input, which predicates can be proved to hold of its output?

The formal inference mechanism is to be specified in a future revision.

15. Continuations and Control Flow

Möbius provides three continuation foundations and one well-known continuation binding for non-local control flow. Because content-addressed definitions have no free variables, these foundations operate entirely at runtime on the continuation — the chain of pending computation — rather than on lexical environments.

12.1 call/cc

call/cc takes a single combiner as argument and calls it with a first-class continuation object representing the current point of execution. This continuation is unlimited — it captures the entire future of the computation. The continuation is a value like any other and may be stored in a tree, returned, or passed to other combiners. It remains valid indefinitely, including after the dynamic extent of the call/cc has exited.

If the continuation is never applied, it has no effect. The combiner may return normally, in which case the result of call/cc is whatever the combiner returns.

12.2 continuation-apply

continuation-apply takes a continuation object and a value, and delivers that value to the captured continuation. This abandons the current computation entirely — control jumps to the point captured by call/cc, and the delivered value becomes the result of that original call/cc expression.

If the continuation crosses guard boundaries, the corresponding guard clauses are invoked during the pass. Applying a continuation that was captured in a dynamic extent that has already exited is permitted. Applying the same continuation multiple times is permitted.

12.3 guard

guard installs entry and exit gamma clauses on a continuation boundary and executes a thunk within that boundary. It merges the roles of Scheme’s guard (exception handling) and dynamic-wind (entry/exit behavior) into a single pattern-based mechanism.

Round:

```
(guard
  (entry (pattern1 handler1)
        (pattern2 handler2)
        ...))
thunk
(exit (pattern1 handler1))
```

```
(pattern2 handler2)
...))
```

Curly:

```
guard {
  entry:
    case pattern1: handler1;
    case pattern2: handler2;
  body: thunk;
  exit:
    case pattern1: handler1;
    case pattern2: handler2;
};
```

Spacy:

```
guard:
  entry:
    case pattern1: handler1
    case pattern2: handler2
  body: thunk
  exit:
    case pattern1: handler1
    case pattern2: handler2
```

Entry clauses. When an abnormal pass (a value delivered via continuation-apply) crosses into this guard boundary from outside, the passed value is matched against the entry clauses in order. If a clause matches, its body executes to handle the pass. If no clause matches, the pass propagates automatically.

Thunk. A zero-argument combiner that executes as the guarded body. The thunk runs within the protection of the guard boundary.

Exit clauses. When an abnormal pass crosses out of this guard boundary to outside, the passed value is matched against the exit clauses. This is the cleanup mechanism — exit clauses run when control leaves the guarded region, whether normally or via continuation.

Guards are the sole mechanism for intercepting non-local control flow. They subsume exception handling (entry clauses matching error values), cleanup (exit clauses performing side effects then propagating), and dynamic-wind entry/exit behavior (entry and exit clauses on the same boundary).

The key edge case: if a guard clause itself signals an abnormal exit, that exit propagates outward past the current guard — a guard does not intercept its own errors.

12.4 The root continuation (continuation-exit)

continuation-exit is a well-known binding — not a separate foundation, but the root continuation that exists when a program starts. It represents “terminate the process.”

Delivering a value to continuation-exit terminates the program. The delivered value must be an integer 0–255 (POSIX exit code). Delivering a non-integer or out-of-range value is an error.

```
;; round
(continuation-apply continuation-exit 0)    ;; exit successfully

// curly
continuation-apply(continuation-exit 0);

# spacy
continuation-apply(continuation-exit 0)
```

Guards installed between the current point and the root continuation are traversed on exit, so cleanup code runs.

If the program’s main expression returns normally (without explicitly calling continuation-exit), the runtime delivers 0 to the root continuation — successful termination.

16. Reader Syntax

The reader converts a character stream into trees. It is not part of the language semantics — it is one possible surface syntax. This section specifies the round (S-expression) reader. The curly and spacy readers (§6.3, §6.4) produce identical trees through different surface conventions.

13.1 Atoms

Integers. A sequence of digits, optionally preceded by - or +. Examples: 42, -7, 0, +3. Positive integers may also be written in hexadecimal with the 0x prefix: 0xFF, 0xC0FF33, 0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8.

Floats. Digits with a decimal point, optionally preceded by a sign, optionally followed by an exponent. Examples: 3.14, -0.5, 1e10, 2.5e-3.

Characters. #\ followed by a single character or a character name. Examples: #\a, #\Z, #\space, #\newline, #\tab. The reader recognizes at least space, newline, tab, and return.

Strings. Delimited by double quotes. Escape sequences: \\, \", \n, \t, \r.

13.2 Hash-identifiers

A # followed by one or more alphanumeric characters or hyphens. Hash-identifiers are identifiers — resolved at registration time like symbols.

Five are pre-defined:

- `#true` — the truthy value.
- `#false` — the sole false value.
- `#nil` — the empty list.
- `#void` — the void singleton, returned by side-effecting operations.
- `#eof` — the end-of-file singleton.

All others are regular identifiers. By convention, a hash-identifier names a singleton.

13.3 Symbols (surface syntax only)

An identifier starts with an alphabetical character or one of `+ - * / < > = ? ! _` and continues until whitespace or a structural delimiter: `( ) { } ; : ' ` " , .`. This rule is the same across all three surfaces (§6.5).

Examples: `foo`, `car`, `+`, `list->string`, `point?`, `encapsulation-type`, `ab-cd`, `box!`.

Symbols are surface syntax only. At registration time, they are resolved to content hashes and disappear. They do not exist as runtime values.

13.4 Parentheses and pairs

`(a b c)` reads as `(cons a (cons b (cons c #nil)))` — a proper list.

`(a . b)` reads as `(cons a b)` — a pair.

`(a b . c)` reads as `(cons a (cons b c))` — an improper list.

Dot notation in expression position is reader shorthand for `cons`, not a distinct expression form.

13.5 Comments

`;` begins a line comment (round surface). `//` begins a line comment (curly surface). `#` begins a line comment (spacy surface).

`#;` is a datum comment in all surfaces — it comments out the next complete expression:

```
(+ 1 #;(ignored) 2)    ;; => 3
```

17. Foundations and Base Library

The vocabulary of Möbius is organized by a single criterion: **can it be written in Möbius?**

Foundations are forms and combinators that require the evaluator or runtime. They cannot be expressed as Möbius programs. Some foundations have equivalent expansions in terms of other foundations — `lambda` in terms of `gamma`, and in terms of `if`. These equivalences are semantic facts the compiler may exploit, not a hierarchy of dependence. The language *chooses* to present all of them as building blocks.

Base library combinators are Möbius programs. They have content hashes and live in the store. They are the first programs written in the language, shipped alongside it. Any programmer could write them.

14.1 Foundations

Core forms (special evaluation rules):

Foundation	Role
gamma	Tree destructuring, binding, catamorphism, combiner construction
if	Conditional — only <code>#false</code> triggers the else branch
and	Short-circuit conjunction. Equivalent to nested if: <code>(and a b c) = (if a (if b c #false) #false)</code>
or	Short-circuit disjunction. Returns first non- <code>#false</code> value. Equivalent to nested if with temporary bindings.
lambda	Single-clause gamma with all-bind patterns. <code>(lambda (a b) body) = (gamma ((,a ,b) body))</code>
begin	Sequencing (round surface; braces in curly, indentation in spacy)
define	Bind a name (top-level: content hash; nested: local binding)
guard	Install entry/exit gamma clauses on a continuation boundary

Data construction and access:

Foundation	Role
cons	Construct a pair
car	First element of a pair
cdr	Second element of a pair
encapsulation-type	Create a capsule type from an integer $< 2^{12}$ (constructor, predicate, accessor)
box	Create a mutable box
unbox	Read box contents
box!	Mutate box contents
list->string	Convert a list of characters to a string
string->list	Convert a string to a list of characters
integer->char	Convert an integer (Unicode code point) to a character

Foundation	Role
char->integer	Convert a character to its integer code point

Continuations:

Foundation	Role
call/cc	Reify the current continuation
continuation-apply	Deliver a value to a captured continuation

Type predicates:

Foundation	Role
integer?	Test if value is an integer
float?	Test if value is a float
char?	Test if value is a character
string?	Test if value is a string
pair?	Test if value is a pair
box?	Test if value is a box
combiner?	Test if value is a combiner
continuation?	Test if value is a continuation

Comparison and arithmetic:

Foundation	Role
eq?	Identity comparison
+, -, *, /	Arithmetic
<, >, =	Comparison

I/O and assertions:

Foundation	Role
display	Write a value to standard output. Returns #void.

Foundation	Role
<code>assume</code>	Runtime assertion. If the first argument is <code>#false</code> , signal failure with the optional second argument as message. If true, returns <code>#true</code> . The failure mechanism is host-specific; the semantics — “if false, stop” — are portable.

Well-known bindings (values bound in the initial environment):

Binding	Role
<code>continuation-exit</code>	The root continuation — delivers an exit code 0–255 to terminate
<code>#true</code> , <code>#false</code> , <code>#nil</code> , <code>#void</code> , <code>#eof</code>	Pre-defined constants

Foundation count: 8 core forms + 11 data + 2 continuations + 8 type predicates + 8 arithmetic/comparison + 2 I/O/assertions = **39 foundations**.

14.2 Base library

Möbius programs shipped with the system. They have content hashes and live in the content store.

Name	Definition	Role
<code>list</code>	<code>(gamma (,args args))</code>	Identity — returns the argument tree. <code>(list 1 2 3)</code> constructs <code>(1 . (2 . (3 . #nil)))</code> via the application rule and returns it unchanged.
<code>not</code>	<code>(gamma ((#false #true) (, _ #false)))</code>	Boolean negation
<code>equal?</code>	Recursive structural comparison using <code>gamma</code> , <code>pair?</code> , <code>eq?</code> , <code>car</code> , <code>cdr</code> , <code>unbox</code> , <code>box?</code> . Pairs compared element-wise. Capsules opaque (only <code>eq?</code>). Boxes compared by current contents.	Deep equality

Name	Definition	Role
continuation-extend	Built from call/cc and continuation-apply. Takes a continuation and a combiner, returns a new continuation such that delivering v applies f to v first, then delivers the result to k.	Continuation composition
capsule-constructor	(gamma ((,type) (car type)))	Extract the constructor from an encapsulation-type result.
capsule-predicate	(gamma ((,type) (car (cdr type))))	Extract the predicate from an encapsulation-type result.
capsule-unwrap	(gamma ((,type) (car (cdr (cdr type)))))	Extract the accessor from an encapsulation-type result.
error	Takes an exit code, a message string, and a tree. Displays the message and tree via display, then delivers the exit code to continuation-exit.	Error reporting and termination

Note on eq? vs equal?: eq? tests identity — whether two values are the same object. For atoms, eq? compares values. For pairs, eq? compares identity. For capsules, eq? compares identity. For boxes, eq? compares cell identity. equal? recurses through pairs and box contents. Capsules are opaque to equal? — two capsules are equal? only if they are eq?.

18. Grammar

15.1 Round surface grammar

```

program      ::= expression*

expression   ::= atom
               | identifier
               | (gamma clause+)
               | (if expression expression expression)
               | (begin expression+)
               | (define identifier expression)
               | (guard guard-body)

```

```

        | (lambda formals expression)      ;; foundation
        | (and expression+)                ;; foundation
        | (or expression+)                 ;; foundation
        | (expression expression*)
clause      ::= (pattern expression)

guard-body ::= (entry clause*) (exit clause*) expression

formals     ::= (identifier*)

pattern     ::= atom                      ;; literal gamma
              | ,identifier                ;; bind
              | ,(identifier)              ;; recurse-and-bind
              | ,_                          ;; wildcard
              | (? identifier ,identifier) ;; predicate guard
              | (pattern . pattern)        ;; pair
              | (pattern*)                 ;; list (shorthand for nested pairs)

atom        ::= integer | float | character | string

integer     ::= [+]? digit+
              | '0x' hex-digit+
float       ::= [+]? digit+ '.' digit* ([eE] [+]? digit+)?
character   ::= '#\' (character-name | any-character)
string      ::= '\"' string-char* '\"'

identifier  ::= symbol | hash-id
symbol      ::= id-start id-continue*
id-start    ::= alpha | '+' | '-' | '*' | '/' | '<' | '>' | '=' | '?' | '!' | '_'
id-continue ::= (any character except whitespace and delimiters: ( ) { } ; : ' ` " ,)
hash-id     ::= '#' (alpha | digit | '-' )+

```

15.2 Curly surface grammar

```

program     ::= statement*

statement   ::= define-stmt | expression ';'

define-stmt ::= 'define' identifier expression ';'
              | 'define' identifier gamma-expr ';'
              | 'define' identifier lambda-expr ';'

expression ::= atom
              | identifier
              | gamma-expr

```

```

    | lambda-expr
    | guard-expr
    | identifier '(' args ')'          ;; application
    | '(' expression op expression ')' ;; infix
    | 'if' '(' expression ')' block
      ('else' block)?
    | '{' statement* expression '}'    ;; begin block

gamma-expr ::= 'gamma' '{' case-clause+ '}'

case-clause ::= 'case' pattern ':' expression ';'
              | 'case' pattern ':' block ';'

lambda-expr ::= 'lambda' '(' identifier* ')' block

guard-expr ::= 'guard' '{'
               'entry' ':' case-clause*
               'exit' ':' case-clause*
               'body' ':' expression ';'
               '}'

block ::= '{' statement* expression '}'

args ::= expression*

op ::= '+' | '-' | '*' | '/' | '<' | '>' | '='
     | identifier          ;; any combiner as infix

pattern ::= (same as round – §15.1)

```

15.3 Spacy surface grammar

```

program ::= line*

line ::= define-line | expression

define-line ::= 'define' identifier ':' expression
              | 'define' identifier ':' gamma-block
              | 'define' identifier ':' lambda-line

expression ::= atom
            | identifier
            | identifier '(' args ')'          ;; application
            | '(' expression op expression ')' ;; infix
            | if-expr

```

```

gamma-block ::= 'gamma' ':' INDENT case-line+ DEDENT

case-line   ::= 'case' pattern ':' expression
               | 'case' pattern ':' INDENT line+ DEDENT

lambda-line ::= 'lambda' '(' identifier* ')' ':' expression

guard-block ::= 'guard' ':' INDENT
                'entry' ':' INDENT case-line* DEDENT
                'exit' ':' INDENT case-line* DEDENT
                'body' ':' expression
                DEDENT

if-expr     ::= 'if' '(' expression ')' ':' INDENT
                line+ DEDENT
                'else' ':' INDENT line+ DEDENT

args        ::= expression*

op          ::= (same as curly – §15.2)

pattern     ::= (same as round – §15.1)

INDENT      ::= (increase in indentation level)
DEDENT      ::= (decrease in indentation level)

```

Note: anonymous lambda/gamma as arguments is not supported in any surface. Use a named define instead.

19. Formal Semantics

This chapter provides the mathematical specification of Möbius evaluation. It is normative: an implementation that disagrees with these rules is incorrect.

19.1 Notation

Let v range over values, e over expressions, ρ over environments (finite maps from de Bruijn indices to values), and T range over trees (the content-addressed representation).

We write $\rho \vdash e \Downarrow v$ to mean “expression e evaluates to value v in environment ρ .”

We write $\rho[i \mapsto v]$ for the environment ρ extended with index i bound to value v .

19.2 Values

```
v ::= n                                (integer)
```

f	(float)
c	(character)
s	(string)
#true #false	(booleans)
#nil	(empty list)
#void	(void)
#eof	(end of file)
(v ₁ . v ₂)	(pair)
(box v)	(mutable box)
(capsule τ v)	(capsule with type id τ)
(combiner clauses ρ)	(closure)
(continuation κ)	(first-class continuation)

19.3 Self-evaluation

$$\frac{}{\rho \vdash n \Downarrow n} \text{ [Atom]}$$

$$\frac{}{\rho \vdash s \Downarrow s} \text{ [String]}$$

$$\frac{}{\rho \vdash \#true \Downarrow \#true} \text{ [Bool-True]}$$

$$\frac{}{\rho \vdash \#false \Downarrow \#false} \text{ [Bool-False]}$$

$$\frac{}{\rho \vdash \#nil \Downarrow \#nil} \text{ [Nil]}$$

19.4 Variable lookup

$$\frac{\rho(i) = v}{\rho \vdash (\text{var } i) \Downarrow v} \text{ [Var]}$$

19.5 Conditional

$$\frac{\rho \vdash e_1 \Downarrow v_1 \quad v_1 \neq \#false \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash (\text{if } e_1 \ e_2 \ e_3) \Downarrow v_2} \text{ [If-True]}$$

$$\frac{\rho \vdash e_1 \Downarrow \#false \quad \rho \vdash e_3 \Downarrow v_3}{\rho \vdash (\text{if } e_1 \ e_2 \ e_3) \Downarrow v_3} \text{ [If-False]}$$

19.6 Gamma (combiner construction)

$$\frac{\text{clauses} = [(p_1, b_1), \dots, (p_n, b_n)]}{\rho \vdash (\text{gamma clauses}) \Downarrow (\text{combiner clauses } \rho)} \quad [\text{Gamma}]$$

19.7 Application

$$\frac{\begin{array}{l} \rho \vdash e_0 \Downarrow (\text{combiner } [(p_1, b_1), \dots, (p_n, b_n)] \rho') \\ \rho \vdash e_1 \Downarrow v_1 \quad \dots \quad \rho \vdash e_k \Downarrow v_k \\ \text{arg} = (v_1 . (v_2 . (\dots . (v_k . \#nil)))) \\ \text{match}(\rho_j, \text{arg}, \text{self}) = \sigma \quad \text{for first matching clause } j \\ \rho'[\sigma][\emptyset \mapsto \text{self}] \vdash b_j \Downarrow v \end{array}}{\rho \vdash (e_0 e_1 \dots e_k) \Downarrow v} \quad [\text{Apply}]$$

where $\text{self} = (\text{combiner } [(p_1, b_1), \dots, (p_n, b_n)] \rho')$

19.8 Pattern matching

Pattern matching is defined by the function $\text{match}(p, v, \text{self})$ which returns a substitution σ (a map from indices to values) on success, or $\perp$ on failure.

$\text{match}(\text{literal}, v, \text{self}) = \{ \}$ if $\text{literal} = v$, $\perp$ otherwise

$\text{match}(\text{bind } i, v, \text{self}) = \{i \mapsto v\}$

$\text{match}(\text{wildcard}, v, \text{self}) = \{ \}$

$\text{match}(\text{catamorphic-bind } i, v, \text{self}) = \{i \mapsto \text{apply}(\text{self}, v)\}$

$\text{match}(\text{guard pred (bind } i), v, \text{self}) =$
 let $p = \text{apply}(\text{pred}, (v . \#nil))$ in
 if $p \neq \#false$ then $\{i \mapsto v\}$ else $\perp$

$\text{match}((p_1 . p_2), (v_1 . v_2), \text{self}) =$
 let $\sigma_1 = \text{match}(p_1, v_1, \text{self})$ in
 if $\sigma_1 = \perp$ then $\perp$
 else let $\sigma_2 = \text{match}(p_2, v_2, \text{self})$ in
 if $\sigma_2 = \perp$ then $\perp$
 else $\sigma_1 \cup \sigma_2$

$\text{match}(p, v, \text{self}) = \perp$ otherwise

Catamorphic termination. Because $(\text{catamorphic-bind } i)$ applies self only to strict structural sub-parts of the matched value, and values are finite trees, catamorphic matching always terminates. This is a structural guarantee — no proof obligation falls on the programmer.

19.9 Arithmetic and comparison

$$\frac{\rho \vdash e_1 \Downarrow n_1 \quad \rho \vdash e_2 \Downarrow n_2}{\rho \vdash (+ e_1 e_2) \Downarrow n_1 + n_2} \text{ [Add]}$$

Analogous rules for $-$, $*$, $/$ (with division-by-zero as error), $<$, $>$, $=$.

19.10 Cons, car, cdr

$$\frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash (\text{cons } e_1 e_2) \Downarrow (v_1 . v_2)} \text{ [Cons]}$$

$$\frac{\rho \vdash e \Downarrow (v_1 . v_2)}{\rho \vdash (\text{car } e) \Downarrow v_1} \text{ [Car]}$$

$$\frac{\rho \vdash e \Downarrow (v_1 . v_2)}{\rho \vdash (\text{cdr } e) \Downarrow v_2} \text{ [Cdr]}$$

19.11 Content-addressing

The *hash* function maps trees to 256-bit identifiers:

$\text{hash} : \text{Tree} \rightarrow \text{Hash}$

```
hash(atom)      = sha256(serialize(atom))
hash(v1 . v2) = sha256(serialize(hash(v1), hash(v2)))
hash(ref h)     = h
```

De Bruijn normalization. Before hashing, a combiner’s body is normalized: all bound variable names are replaced by positional indices. Two combinators that differ only in variable names produce the same hash.

Registration. A top-level definition (`define name expr`) evaluates `expr`, normalizes the result, computes its hash, stores the tree, and records the `name`→`hash` mapping.

19.12 TODO

The following formal rules remain to be specified:

- `begin`, `define` (nested), sequencing
- `and`, `or` (short-circuit, in terms of `if`)
- `box`, `unbox`, `box!` (mutable state)
- `call/cc`, continuation-apply, guard (control flow)
- encapsulation-type, capsule operations
- `display`, `assume`, `xeno`

- Error semantics (what happens when `car` is applied to an atom, when no gamma clause matches, when division by zero occurs)

20. Reference Test Suite

The reference test suite is a set of transcript files — Möbius source paired with expected output. A conforming implementation passes all tests.

The test suite is normative: where the prose specification and the test suite disagree, the test suite defines the intended behavior (and the prose should be corrected).

20.1 Test categories

File	What it tests
<code>basics.md</code>	Arithmetic, booleans, conditionals, string literals, <code>cons/car/cdr</code> , equality
<code>recursion.md</code>	Self-recursion via gamma, mutual recursion, catamorphic <code>sum/product/length</code> , GCD
<code>advanced.md</code>	Boxes, capsules, <code>call/cc</code> , lambda, higher-order functions
<code>catamorphic-arith.md</code>	Tree-walking arithmetic evaluator using catamorphic <code>(a)</code> patterns
<code>closures.md</code>	Closures, currying, composition, accumulators, fold-right
<code>patterns.md</code>	Multi-argument gamma, nested destructuring, predicate guards, wildcards

20.2 Test format

Each test is a fenced code block followed by expected exit code and expected output:

```
## Test name

``scheme
(display (+ 3 7))
``
```

Expected exit code: 0
Expected output: 10

20.3 Transcript tests for the toolchain

Toolchain behavior is tested via bash transcript files that exercise `bb` commands end-to-end:

File	What it tests
00-beyond-babel.md	bb store init, bb add, bb edit roundtrip
01-triple-evolution.md	Iterative refinement with --derived-from, bb show, bb status

These tests are content-addressed once they live in the store. A conforming bb implementation passes them.

Part III — The Store

21. Content Model

Möbius separates **content** (immutable, content-addressed) from **naming** (mutable, versioned).

21.1 The content store

The content store is an immutable mapping from hashes to trees.

store : Hash → Tree

Every distinct tree has a unique hash, computed from its structure — atoms, pairs, and references to other hashes. Once stored, a tree cannot be changed. Its hash is its identity.

22. Naming

Names are views into mappings stored alongside combinators. There is no separate “registry” data structure. The name→hash relationship is emergent: each combiner in the content store may have one or more **mappings** — content-addressed files that associate de Bruijn indices with human-readable names in a specific language.

mapping : (Index → Name) × Language × Hash

The same combiner hash may have mappings in multiple languages. The same function named differently in different languages points to the same hash:

"odd?" → 0x7a3f... (English mapping)

"impair?" → 0x7a3f... (French mapping)

"فردی؟" → 0x7a3f... (Arabic mapping)

A **name index** is computed on demand by scanning all mappings in the store. It is a derived view, not stored state. The name index maps the name at position 0 of each

mapping (the combiner’s own name) to its content hash. When multiple combiners share a name, disambiguation uses the short hash suffix.

Naming is the **image** — the living, evolving view into the content store. Adding a new mapping for a combiner does not change the combiner’s tree or hash. Mappings are independently content-addressed, timestamped, and authored. A mapping is a real intellectual contribution — naming is meaning.

23. Registration

Registration converts surface syntax into content-addressed form:

1. **Parse** surface syntax into an AST with names.
2. **Resolve** each name against the current name index, obtaining a hash.
3. **Replace** names with hashes, producing a tree of atoms, pairs, and hash references.
4. **Compute** the hash of the resulting tree.
5. **Store** the tree in the content store (if not already present).
6. **Store** the mapping (name→index associations and language) alongside the combiner.

After registration, the original names are gone from the content. Only hashes remain. The mapping preserves the names as a separate, independently content-addressed artifact.

The dependency graph between top-level definitions is strictly a directed acyclic graph. Definition A may reference definition B only if B is already registered. Top-level mutually recursive definitions are not yet supported (Chapter 33).

Note: the reader and the registration pipeline may internally represent names as symbols. Symbols are an implementation detail of the toolchain, not a user-visible value type. The constraint “no symbols as values” applies to the content store and to runtime, not to intermediate representations.

24. Combiner Structure

A combiner in the content store is a tree containing:

- **Atoms:** integers, floats, characters, strings — self-hashing.
- **Pairs:** structure — hash is computed from the hashes of car and cdr.
- **Hash references:** pointers to other content (foundations, constants, other combiners).
- **Bound variables:** positions introduced by patterns and used within the same combiner.

Bound variables are local to a combiner and represented by position (de Bruijn indices or equivalent), not by name. This ensures that alpha-equivalent combiners hash to the same value.

24.1 Foundations in the content store

Foundations are forms and combinators built into the runtime with reserved hashes — integers smaller than 2^{12} , known to all implementations. Foundations are not stored in the content store — they are intrinsic.

25. Lineage

Lineage records the history of a combiner: who created it, when, what it derived from, and what checks it satisfies. Lineage is stored in the `lineage/` subdirectory of each combiner's directory.

25.1 Lineage records

A lineage record is a content-addressed file containing:

```
((author . "name")
 (committed . "2026-04-01T14:30:00Z")
 (relation . "add")
 (derived-from . "a4f7e2..." ;; optional - hash of parent combiner
 (checks . ("c8a31f..." "7b2e04...") ;; optional - check hashes
 (note . "...") ;; optional - human-readable note
 (replaces . "...") ;; optional - hash this supersedes
```

25.2 WIP and committed

A lineage record begins as **WIP** (work in progress) when created by `bb add`. It becomes **committed** when promoted by `bb commit`. The distinction is a lifecycle marker:

- WIP lineage has a created timestamp.
- Committed lineage has a committed timestamp and is eligible for `bb anchor`.

25.3 Relations

The relation field describes how the combiner came into being:

Relation	Meaning
add	New content, no derivation
fork	New line of work derived from existing
fix	Bug fix
refine	Improvement without changing behavior
translate	Same algorithm, different human language mapping
extend	Added functionality
rewrite	Complete reimplementation
refactor	Propagation of a dependency change
commit	Promotion from WIP to committed

25.4 The lineage graph

The derived-from pointers form a directed acyclic graph of derivation. This graph is the structural record of how knowledge evolved. Every edge is timestamped and authored. The graph is append-only — edges cannot be removed once committed.

The lineage graph is distinct from the dependency DAG (which records which combinators reference which other combinators in their de Bruijn trees). Lineage records *how things came to be*. Dependencies record *what things use*.

26. Checks

A check is a combiner that takes one argument — the combiner under test — and verifies properties of it using `assume`. Checks are the specification language of Möbius: a well-written check suite is a problem statement.

Round:

```
(define ~check-gcd-base-cases
  (lambda (gcd)
    (begin
      (assume (= 6 (gcd 6 0)) "gcd(6,0) should be 6")
      (assume (= 7 (gcd 0 7)) "gcd(0,7) should be 7")
      (assume (= 1 (gcd 1 1)) "gcd(1,1) should be 1"))))

(define ~check-gcd-commutative
  (lambda (gcd)
    (begin
      (assume (= (gcd 48 18) (gcd 18 48)) "gcd must be commutative")
      (assume (= (gcd 100 75) (gcd 75 100)) "gcd must be commutative"))))
```

Curly:

```
define ~check-gcd-base-cases lambda (gcd) {
  assume((gcd(6 0) = 6) "gcd(6,0) should be 6");
  assume((gcd(0 7) = 7) "gcd(0,7) should be 7");
  assume((gcd(1 1) = 1) "gcd(1,1) should be 1");
};
```

Spacy:

```
define ~check-gcd-base-cases: lambda (gcd):
  assume((gcd(6 0) = 6) "gcd(6,0) should be 6")
  assume((gcd(0 7) = 7) "gcd(0,7) should be 7")
  assume((gcd(1 1) = 1) "gcd(1,1) should be 1")
```

Why lambda, not gamma. A check takes one argument — the candidate combiner — and names it. There is no dispatch on shape. Lambda is the right tool: one parameter, one name, the parameter name documents what the check expects.

Why the combiner under test is an argument, not a name. The check receives the candidate as a value, not as a name bound in the environment. This makes checks portable: the same check can verify any combiner that claims to satisfy the specification, in any store, under any name, in any language. The check’s hash depends only on what it tests, not on who it tests.

Naming convention. Check names begin with `~check-`. This is a convention enforced by the `bb` toolchain, not a language rule. The `~` prefix prevents collision with ordinary combinators and signals to `bb add` that the combiner is a check, not content.

Check execution. `bb check` applies each check combiner to the combiner under test:
(`~check-gcd-base-cases gcd`)

If every `assume` passes, the check passes. If any `assume` fails, the check fails and reports which assertion failed. `bb check` runs all checks associated with a combiner and reports results.

Checks are content-addressed. Each check is a combiner in the store with its own hash, its own mappings, its own lineage. A check can be forked, translated, reviewed, and curated like any other combiner. The association between a combiner and its checks is recorded in the combiner’s lineage — the check hashes are listed there, not embedded in the combiner’s tree.

Checks and ZKPs. Because a check takes the combiner under test as an argument, the ZKP statement is clean: “there exists a tree `T` such that `sha256(serialize(T)) = H` and `check(T)` passes.” One application, no environment setup, no name resolution in the proof circuit.

27. Sealed Timestamps

Content may be committed and timestamped without disclosure. A combiner’s hash is computed from its de Bruijn tree and stored locally. `bb anchor` requests an Open-Timestamps proof — a Bitcoin-anchored timestamp that makes priority cryptographic, not just local. The content stays in the local store. Nobody sees it. Content only leaves when explicitly pushed via `bb remote push` or `bb remote sync`.

The priority proof and the content disclosure are separate acts. The sealed envelope, made mathematical. If the author ever needs to prove they made it first, they reveal the content and the hash matches what the timestamp already proved.

A store that accommodates sealed hashes accumulates **dark matter** — timestamps that anchor facts nobody can see. This is not a bug. It is a structural feature. The realistic population of knowledge workers includes people who need priority proof without disclosure: industrial researchers, solo inventors, small teams in competitive fields. Omitting them maps a fantasy commons, not a real one.

28. Zero-Knowledge Proofs

A sealed hash proves existence and priority. It does not prove capability. Zero-Knowledge Proofs close that gap without breaking the seal.

A ZKP allows the holder of sealed content to publish a cryptographic proof that their hidden combiner passes a given check suite — the same `bb` check infrastructure every other participant uses — without revealing the content itself. The proof binds a specific sealed hash to a specific content-addressed check suite: “the combiner at hash H satisfies checks C.” The silent hash becomes a **verified capability**.

Proof placement. Proofs are content-addressed artifacts stored in `proofs/`, adjacent to the combiner they attest to, with their own timestamps. They are not fields in the mapping or the lineage record. This keeps naming clean and makes proofs first-class objects that the Curator can query across the store.

Two zones of sealed content. Not all sealed hashes are equal. The store distinguishes:

Sealed without ZKP. A hash and a timestamp. No behavioral properties are visible. Structural similarity detection (`bb search --near`) cannot operate — it requires the de Bruijn tree, which is not public. If it were, the seal would be broken. The Curator is blind. This is the honest cost of accommodating sealed content.

Sealed with ZKP. The content is hidden but the capabilities are visible — specifically, which check suites are satisfied. If three sealed hashes each have ZKP proofs against the same check suite, the Curator sees behavioral convergence: three implementations satisfying the same specification, identifiable by what they satisfy rather than what they contain. The check suite is the shared vocabulary that lets sealed work participate in the graph without disclosure.

What does not change. The content store, the de Bruijn normalization, the hashing, the naming layer — none of these are affected by ZKPs. The proof is an overlay, not a modification. A store without any ZKP infrastructure is a valid Möbius store. ZKPs are a companion technology that strengthens the Privateer’s participation without requiring every implementation to support them.

Part IV — The Toolchain

29. `bb` Command Reference

`bb` is the Möbius toolchain — it replaces `git` for knowledge work. Every command operates on the content-addressed store.

29.1 Content authoring

Command	Usage	Description
bb add	bb add [--derived- from=<ref>] [-- relation=<type>] <file\ ->	Parse, normalize, store, and create mapping. Stages as WIP.
bb edit	bb edit <ref> [lang]	Open combiner in \$EDITOR, re-add on save. Roundtrips through denormalization and renormalization.
bb commit	bb commit [name... \ - --all]	Promote WIP lineage to committed.
bb refactor	bb refactor <root> <old> <new> [<at>]	Replace old hash with new hash in root's dependency tree. Cascades.

29.2 Verification

Command	Usage	Description
bb check	bb check <ref>	Run all checks for ref and its dependencies.
bb review	bb review <ref>	Mark combiner as reviewed. Attestation is permanent and authored.
bb validate	bb validate	Verify store integrity — recompute all hashes and compare.
bb diff	bb diff <ref> <ref>	Pretty-printed structural diff with ANSI colors.

29.3 Navigation

Command	Usage	Description
bb show	bb show <ref>	Display combiner definition with doc and mapping.
bb search	bb search <query>	Search combiner names and content.
bb tree	bb tree <ref>	Show dependency DAG downward.
bb caller	bb caller <ref>	Show reverse dependency DAG.

Command	Usage	Description
bb resolve	bb resolve <ref>	Resolve ref to full name@hash@lang@mappingHash spec.
bb log	bb log [ref]	Show lineage timeline.
bb status	bb status	Show working state — committed, WIP, reviewed.

29.4 Evaluation

Command	Usage	Description
bb eval	bb eval <expression>	Evaluate a single expression.
bb run	bb run <ref> [args...]	Evaluate a registered combiner with arguments.
bb repl	bb repl	Interactive Seed session. Loads store combiners.
bb print	bb print <ref>	Output Chez Scheme library with all dependencies.

29.5 Distribution

Command	Usage	Description
bb anchor	bb anchor <remote>	Push committed combiners to remote, create timestamp proofs.
bb remote add	bb remote add [-- read-only] <name> <url>	Add a remote store endpoint (file://, git+ssh://, git+https://).
bb remote remove	bb remote remove <name>	Remove remote.
bb remote list	bb remote list	List configured remotes.
bb remote push	bb remote push <name>	Push committed combiners to remote.
bb remote pull	bb remote pull <name>	Pull combiners from remote.
bb remote sync	bb remote sync	Pull and push all configured remotes.

29.6 Housekeeping

Command	Usage	Description
bb store init	bb store init [path]	Create a new mobius-store.
bb store info	bb store info	Show store statistics.
bb worklog	bb worklog <ref> [msg]	View or add timestamped work log entries.

29.7 Ref syntax

A <ref> identifies a combiner. The format is flexible:

Form	Example	Meaning
name	gcd	Lookup by name in the name index
hash prefix	a4f7e2	Unique hash prefix match (minimum 4 chars)
name@lang	gcd@fr	Name with language constraint
hash@lang	a4f7e2@kab	Hash with language constraint
name@hash@lang	gcd@a4f7e2@fr	Fully disambiguated
name@hash@lang@map	gcd@a4f7e2@fr@3c2073	Fully specified with mapping hash

When multiple combiners share a name, the name index disambiguates as name@shortHash. When a bare name is ambiguous, the most recently timestamped combiner wins.

29.8 Future commands (not yet implemented)

Command	Usage	Description
bb prove	bb prove <ref> -- checks=<check-refs>	Generate ZKP proof that sealed combiner passes checks.
bb verify	bb verify <ref>	Verify ZKP proof for a sealed combiner.

30. Workflows

30.1 The basic loop

The minimum viable workflow is two roles: Coordinator and Author.

1. Coordinator writes checks:
bb add checks.scm
bb commit --all
2. Author writes implementation:
bb add solution.scm
bb check solution
bb commit solution
bb anchor origin

30.2 Edit-check-commit

The bb edit workflow for iterative refinement:

```
bb edit gcd          # opens in $EDITOR with doc, definition, and checks
                    # on save: parse → eval → run checks
                    #   all pass → re-add to store
                    #   any fail → (d)iscard, (w)orklog, (r)e-edit
bb commit gcd
```

30.3 Fork and derive

Building on someone else's work:

```
bb edit existing-fn  # read the existing implementation
# write your version in a new file
bb add --derived-from=existing-fn --relation=extend new-fn.scm
bb check new-fn
bb commit new-fn
```

30.4 Refactor propagation

When a dependency is updated:

```
bb diff old-dep new-dep      # understand what changed
bb check new-dep             # verify new version passes checks
bb refactor my-fn old-dep new-dep # propagate through dependency tree
bb commit --all
```

30.5 The Privateer workflow

Priority without disclosure:

```
bb add secret-algo.scm      # hash computed locally
bb commit secret-algo       # committed locally
bb anchor origin            # OTS timestamp anchored in Bitcoin
# content stays local – never pushed
# later, optionally:
```

```
bb prove secret-algo --checks==check-spec # ZKP proof (future)
bb remote push origin --proofs-only       # push only the proof
```

30.6 Multilingual contribution

Adding a mapping in another language:

```
bb add gcd.kab.scm kab          # same algorithm, Tamazight names
bb show gcd@kab                 # verify the mapping reads correctly
bb commit --all
```

Part V — Horizon

31. Oblivious Execution

Oblivious execution — running a computation so that the server performing it cannot observe the code, the inputs, or the intermediate states — is **orthogonal** to Möbius. Möbius is a repository: storage, naming, lineage, verification. Oblivious execution is about runtime privacy. A valid Möbius store requires no oblivious execution support. Oblivious execution requires no content-addressed store.

They are documented together because they create a chain that neither provides alone.

31.1 The chain

1. **The Coordinator** writes check suites that define what “correct” means for a problem. The checks are content-addressed and public.
2. **The Privateer** commits a combiner that passes those checks. The content is sealed — only the hash and the OpenTimestamps proof are public. Via a Zero-Knowledge Proof (Chapter 28), the Privateer publishes a cryptographic attestation that the sealed content passes the check suite, without revealing the content.
3. **The Citizen** needs to use the Privateer’s capability on private data — tax calculation, medical scoring, vote verification. The Citizen can verify the ZKP proof: the sealed combiner does what the checks require. But the Citizen cannot see the code, and the Privateer must not see the data.
4. **Oblivious execution** resolves this. Using techniques such as Oblivious RAM (ORAM), Trusted Execution Environments (TEEs), or Multi-Party Computation (MPC), the sealed combiner runs on the Citizen’s private data in a context where the executing machine sees neither the code logic nor the input data. The result is delivered to the Citizen.

5. **The Reviewer** can audit the ZKP proof independently. The lineage is timestamped. The check suite is public. The proof is content-addressed in `proofs/`. Every step is verifiable without requiring disclosure of the sealed content.

31.2 What Möbius provides

Möbius does not implement oblivious execution. It provides the infrastructure that makes oblivious execution *trustworthy*:

- **Content-addressed check suites.** The checks the ZKP attests to are immutable and retrievable by hash. They cannot be swapped after the proof is published.
- **Timestamped lineage.** The Privateer's sealed claim has a verifiable date. The ZKP proof has a verifiable date. The order of events is established cryptographically, not by assertion.
- **Proof as first-class artifact.** The ZKP proof lives in `proofs/` with its own hash, its own timestamp, its own lineage. It can be reviewed, forked, and curated like any other artifact in the store.
- **Behavioral convergence detection.** Sealed hashes with ZKP proofs participate in behavioral indexing — the Curator can discover that multiple sealed combinators satisfy the same check suite, revealing convergence without disclosure. Structural similarity detection (`bb search --near`) requires the de Bruijn tree and cannot operate on sealed content.

31.3 What Möbius does not provide

- Runtime isolation. The `bb` toolchain does not sandbox execution.
- Oblivious memory access patterns. These require hardware support (TEEs) or cryptographic protocols (ORAM, MPC).
- A specific ZKP proof system. The choice of proof system (zk-SNARKs, zk-STARKs, Bulletproofs) is an implementation decision, not a language decision.

31.4 Why it matters

The Privateer + ZKP + Citizen + Oblivious Execution chain is the strongest argument for Möbius as infrastructure rather than a language. It demonstrates that the content model — immutable trees, content-addressed checks, timestamped lineage, separation of content from naming — serves use cases far beyond programming. The same architecture that lets a Kid in Tizi Ouzou name a function in Tamazight also lets a Citizen verify a sealed algorithm without seeing its source. The infrastructure is the same. The personas are different. The hash doesn't care.

32. Atlas Stoa

Atlas Stoa is the read-only aggregation layer above the content store. It is not yet implemented. This chapter records the design direction.

Atlas Stoa ingests multiple Möbius stores and provides queryable indices across them. It does not modify any store — it is a view, not an authority.

32.1 Three embedding axes

Atlas Stoa indexes combinators across three axes:

Structure. The de Bruijn tree shape. Structural similarity detection via SimHash or locality-sensitive hashing. Two combinators that solve the same problem differently may have similar structure.

Behavior. Which check suites a combiner satisfies. Behavioral indexing groups combinators by what they do, not how they do it. This axis is enriched by ZKP proofs — sealed combinators with proofs participate in behavioral indexing without disclosure.

Language. The human-language mappings. A combiner with mappings in Tamazight, French, and English is discoverable from any of those languages.

32.2 Near-duplicate detection

`bb search --near` queries the structural axis. Two combinators with high SimHash similarity and no derived-from edge between them are flagged — not as accusation, but as a prompt: “did you mean to derive from this?”

32.3 Implementation direction

LOUDS-encoded tries for prefix search over hashes. SSTables for sorted key-value storage of index data. ANN (approximate nearest neighbor) indexing for the structural and behavioral axes.

Atlas Stoa operates on disclosed content only. Sealed hashes without ZKP proofs are invisible to all three axes. Sealed hashes with ZKP proofs participate in the behavioral axis only.

33. Open Questions

The following are identified for future revisions:

1. **Predicate inference.** The compiler infers predicates from known properties of foundations — `car` requires `pair?`, `+` requires `integer?` or `float?`. Gamma clause structure propagates these predicates. The programmer may add assert hints. The formal specification of the inference mechanism, the set of foundation predicate signatures, and the interaction between capsule-level inference and tree-level predicates remain to be detailed.
2. **Effects.** Effects in Möbius are named patterns over existing continuation foundations, not new foundations. `raise` (one-shot, no resumption), `raise-continuable` (one-shot resumption), and `coroutines` (multi-shot, sequential resumption) are defined in terms of `guard`, `call/cc`, `continuation-apply`, and `gamma`. The compiler recognizes these named patterns and optimizes accordingly: `raise` compiles to a jump, `raise-continuable` to a call, `coroutines` to stack switching. The formal definitions remain to be specified.

3. **eval semantics.** Without symbols as values and without quote, eval in the traditional Lisp sense is not possible. What remains is hash lookup: given a content hash, retrieve and execute the corresponding combiner from the content store. A possible foundation integer->combiner would take a hash (an integer) and return the corresponding combiner. Open questions: Is this a foundation, or is it implicit in application? What happens when a hash is not in the content store? What are the security implications of arbitrary hash execution?
4. **Error model.** What happens when a gamma fails (no clause matches)? When car is applied to an atom? When division by zero occurs? The spec needs to define: what an error value is (likely a capsule), how errors are raised (likely via continuation-apply to the nearest guard), and how they interact with guard. This is critical for implementers.
5. **Naming layer representation.** How are mappings organized and discovered across federated stores? The current implementation scans all mappings on demand to build a name index. For large stores, this may require indexing. How do multiple stores compose their name indices? Store modes: local-only, federated, centralized.
6. **Concurrency.** Möbius's tree-in-tree-out model and immutable content store are natural fits for concurrent and distributed computation. CSP-style channels (read-channel, write-channel, select) are a candidate model. The interaction between concurrency, boxes, and continuations needs careful specification.
7. **Cycle detection for equal? on boxes.** Self-referential boxes are permitted. equal? on cyclic box structures is currently undefined behavior. Should implementations be required to detect cycles?
8. **I/O model.** display is a foundation but I/O is otherwise unspecified. How do file handles, network sockets, and other resources interact with the content-addressed model? Are they capsules wrapping OS handles?
9. **Top-level mutual recursion.** How should the toolchain handle mutually recursive top-level definitions? It could detect mutual groups and bundle them into a single content-addressed unit whose hash covers the entire group. Open questions: How are stale references handled when one member is updated without the others? Should tooling warn about broken mutual groups? What is the interaction with the dependency DAG?
10. **Capsule type ID derivation.** How should programmers choose type identifiers to avoid collisions? Possible strategies include deriving IDs deterministically from a hash of the defining module's content and a local name. A recommended derivation scheme would reduce accidental collisions.
11. **ZKP proof format and verification.** What proof system is used? How are proofs verified without the content? What is the interaction between the proof and the check suite — does the proof attest to a specific check suite hash, or to a property expressed independently? How does proof size scale with combiner complexity?

These are implementation questions, not language questions, but they determine whether §1.7 is practical.

12. **Oblivious execution integration.** Oblivious execution (ORAM, TEEs, MPC) is orthogonal to Möbius — it is a runtime concern, not a storage concern. But the Privateer + Citizen chain (verified capability via ZKP, private data via oblivious execution) is a primary use case for the infrastructure. What interfaces, if any, should the bb toolchain provide to support oblivious execution? Is this a `bb run -oblivious` flag, a separate tool, or entirely outside scope? See Chapter 31.
13. **Metaprogramming as tree inspection.** Three questions that appear opposite converge on the same answer: (a) how to do metaprogramming without reintroducing `vau`, (b) how to statically prove program properties with the same language used to program, (c) how to keep the result content-addressable and multilingual. The answer may be: trees all the way down. A stored combiner is already a de Bruijn tree — it is data. A combiner that takes another combiner’s tree as input and pattern-matches on its structure is metaprogramming. No `vau` needed because no unevaluated operands are needed — the tree is already in the store, already normalized, already data. The question is whether this is expressive enough to be useful.
14. **Static checks vs dynamic checks.** A check that receives a combiner and calls it is a dynamic assertion (runtime verification). A check that receives a combiner’s de Bruijn tree and inspects its structure is a static assertion (registration-time verification). Both are `lambdas`. Both are content-addressed. Both live in the store. The difference is what they receive — the closure or the tree.

Static checks can verify structural properties: “does this combiner have a base case for `#nil`?”, “does every recursive call go through catamorphic `bind`?”, “does the combiner reference only foundations and declared dependencies?”. These are tree-shape predicates — pattern matching on the de Bruijn form.

Semantic static checks are harder: “does this combiner return a sorted list?” requires reasoning about the meaning of the tree, not just its shape. The boundary between structural checks (tractable), bounded semantic checks (possible for restricted domains like arithmetic and termination), and general semantic checks (undecidable) needs to be mapped. `assume` is the escape hatch: where static verification stops, the programmer asserts and the compiler trusts. This is the same position as Agda’s `postulate` — honest about where the proof ends.

The ZKP story benefits directly: proving a static check over a sealed tree is cheaper than proving an execution trace, because the circuit needs only tree traversal, not the full evaluator. If the boundary between static and dynamic checks is well-drawn, the Privateer can prove structural properties cheaply and reserve execution-trace proofs for behavioral claims.

15. **Parameterization of infrastructure.** Three aspects of the system are currently hardcoded and should become parameters: (a) the anchor target — `bb anchor` currently assumes Bitcoin via `OpenTimestamps`, but the architecture should ac-

commodate multiple anchor chains; (b) the hashing algorithm — SHA-256 is the current choice, but the hash function should be a parameter so the store can migrate if SHA-256 is weakened; (c) the ZKP proof format — proofs/ should accept multiple proof systems (zk-SNARKs, zk-STARKs, Bulletproofs) for the same claim. All three follow the same principle: the store records *what was used*, the choice is the user’s. Same pattern as mappings/ being language-agnostic. The design question is how to encode the parameter choice without breaking content-addressability — if two stores use different hash functions, their hashes for the same tree will differ.

This document records the design of Möbius as understood in April 2026. It is a working specification, not a final standard. The “O” in RORM reflects this: it is the revision before revision, the seed before the tree.

Annexes

Annex A. Ellipsis (Possible Future Addition)

The ellipsis . . . in a pattern would mean “zero or more repetitions of the preceding sub-pattern.” Each . . . would add one depth level to the bindings it contains. In the body, . . . would expand those bindings at the corresponding depth. This section sketches the design; it is not part of the current language.

A.1 Basic iteration — depth 1

```
;; collect all elements
((gamma ((,a ...) (,a ...)))
 (list 1 2 3))
;; => (1 2 3)

;; transform each element
((gamma ((,a ...) ((* 2 a) ...)))
 (list 1 2 3))
;; => (2 4 6)
```

a is bound at depth 1. In the body, (* 2 a) . . . iterates over a.

A.2 Parallel bindings

```
((gamma (((,a ,b) ...) ((+ a b) ...)))  
 (list (list 1 10) (list 2 20) (list 3 30)))  
;; => (11 22 33)
```

Mismatched lengths are an error.

A.3 Nested ellipsis — depth 2

```
;; flatten a list of lists  
((gamma (((,a ...) ...) (,a ... ...)))  
 (list (list 1 2) (list 3 4) (list 5 6)))  
;; => (1 2 3 4 5 6)  
  
;; double every element, preserving structure  
((gamma (((,a ...) ...) ((* 2 a) ... ...)))  
 (list (list 1 2) (list 3 4)))  
;; => ((2 4) (6 8))  
  
;; double every element, then flatten  
((gamma (((,a ...) ...) ((* 2 a) ... ...)))  
 (list (list 1 2) (list 3 4)))  
;; => (2 4 6 8)
```

A.4 Depth rules

- Each ... in a pattern adds one depth level.
- In the body, each ... consumes one depth level.
- A depth-0 binding under ... is repeated at each iteration.
- Multiple bindings at the same depth must have the same length.
- Depth mismatch is an error.

Restrictions: ... only at the tail of a list pattern. (,a ... ,b) is not allowed. (,a ... ,rest) is not supported.

Interaction with catamorphism: Ellipsis and ,(x) are orthogonal. They can appear in the same gamma.

Annex B. Predicate-Driven Optimization

Predicate information drives compiler optimization:

Representation narrowing. A value satisfying `uint4?` needs only 4 bits.

Pointer tagging. Known predicate constraints free up tag bits.

Dead code elimination. Gamma clauses ruled out by predicates are unreachable.

Check elimination. Runtime checks implied by known predicates are redundant.

Specialization. Known predicates enable specialized code paths.

Annex C. Representation Inference

The only compositional data constructor is `cons`. The compiler infers optimal machine representations from predicate knowledge:

Byte vectors. A list of `uint8?` values with known length $\rightarrow$ contiguous bytes.

Arrays. Homogeneous known-length list $\rightarrow$ flat array with $O(1)$ indexed access.

Structs. Fixed-position pair structure with known types $\rightarrow$ struct with fixed offsets.

Hash tables. Association list consistently accessed by key $\rightarrow$ amortized $O(1)$ lookup.

Capsules are the natural boundary where representation choices stabilize. The capsule's opacity guarantees no external code depends on internal layout, freeing the compiler to change representations without breakage.

Annex D. Knowledge Economy

Reference: Aaron Hsu, "Programming Obesity: A Code Health Epidemic" (FnConf 2019)

Hsu proposes three axes for evaluating programming systems: simplicity (structural complexity), speed (execution performance), and economy (how much knowledge scales to a broad set of problems).

Economy is measured as the ratio of domain-specific names to total names in a program. Fewer unique concepts $\rightarrow$ less code $\rightarrow$ faster execution $\rightarrow$ more knowledge reuse. These compound rather than trade off.

Möbius is designed for high economy: one data type (tree), one mechanism (gamma), ~39 foundations. No frameworks, no DSLs, no separate type language. The programmer's attention is spent on the domain problem, not on systems rumination.

Annex E. Abacus — An Arithmetic Evaluator

This annex demonstrates catamorphic match on a concrete problem: evaluating arithmetic expression trees. The entire evaluator is one gamma with no explicit recursion.

E.1 Representation

Expression trees use string tags as node labels:

```
;; round
(list "add" 1 (list "mul" 2 3))      ;; 1 + (2 * 3) = 7
(list "sub" (list "add" 10 5) 3)      ;; (10 + 5) - 3 = 12
(list "mul" (list "add" 1 2) (list "add" 3 4)) ;; (1 + 2) * (3 + 4) = 21
```

A number is a leaf. A tagged list ("op" left right) is a branch. This is a tree — cons all the way down.

E.2 The evaluator

Round:

```
(define abacus
  (gamma ((? integer? ,n) n)
    (("add" ,(left) ,(right)) (+ left right))
    (("sub" ,(left) ,(right)) (- left right))
    (("mul" ,(left) ,(right)) (* left right))
    (("div" ,(left) ,(right)) (/ left right))))

(abacus (list "add" 1 (list "mul" 2 3)))
;; => 7

(abacus (list "mul" (list "add" 1 2) (list "add" 3 4)))
;; => 21
```

Curly:

```
define abacus gamma {
  case (? integer? ,n): n;
  case ("add" ,(left) ,(right)): (left + right);
  case ("sub" ,(left) ,(right)): (left - right);
  case ("mul" ,(left) ,(right)): (left * right);
  case ("div" ,(left) ,(right)): (left / right);
};

abacus(list("add" 1 list("mul" 2 3)));
// => 7
```

Spacy:

```
define abacus: gamma:
  case (? integer? ,n): n
  case ("add" ,(left) ,(right)): (left + right)
  case ("sub" ,(left) ,(right)): (left - right)
  case ("mul" ,(left) ,(right)): (left * right)
  case ("div" ,(left) ,(right)): (left / right)

abacus(list("add" 1 list("mul" 2 3)))
# => 7
```

E.3 How it works

The key is ,(left) and ,(right). These are catamorphic binds — before the body runs, abacus is applied recursively to each subtree.

Trace of (abacus (list "add" 1 (list "mul" 2 3))):

1. Input matches ("add" ,(left) ,(right)).
2. left: apply abacus to 1. Matches (? integer? ,n). Returns 1.
3. right: apply abacus to ("mul" 2 3). Matches ("mul" ,(left) ,(right)).
 - left: apply abacus to 2. Returns 2.
 - right: apply abacus to 3. Returns 3.
 - Body: (* 2 3) → 6.
4. Body: (+ 1 6) → 7.

The programmer writes no recursive calls. The fold is declared in the pattern. The body only sees already-computed values.

E.4 Extending the evaluator

Adding negation — a unary operation:

Round:

```
(define abacus
  (gamma ((? integer? ,n) n)
    (("add" ,(left) ,(right)) (+ left right))
    (("sub" ,(left) ,(right)) (- left right))
    (("mul" ,(left) ,(right)) (* left right))
    (("div" ,(left) ,(right)) (/ left right))
    (("neg" ,(inner))          (- 0 inner))))

(abacus (list "neg" (list "add" 1 2)))
;; => -3
```

Each new operation is one clause. The catamorphism handles the recursion automatically — ,(inner) evaluates the subtree before the body sees it.

E.5 What this demonstrates

- **One mechanism.** gamma handles dispatch, destructuring, recursion, and result construction. No visitor pattern, no interpreter loop, no recursive function definition.
- **The pattern is the program.** The shape of the clause *is* the specification of what the evaluator does. Reading the pattern tells you the input structure; reading the body tells you the output.
- **Catamorphism terminates.** Because ,(x) only recurses into strict sub-parts of the matched value, the evaluator terminates on any finite expression tree. This is a structural guarantee, not a proof obligation.
- **Economy.** The evaluator uses 6 names from Möbius (gamma, integer?, +, -, *, /) and 5 from the domain ("add", "sub", "mul", "div", "neg"). Half the program is the problem; half is the tool.

Annex F. Historical Changelog

F.1 Changes from Draft 4 to Draft 5 (February 2026)

- **match renamed to gamma.** The core mechanism is now called gamma, reflecting its role as the sole combiner constructor. All sections, examples, and grammar updated.
- **Quote and quasiquote removed.** quote, quasiquote, unquote, unquote-splicing eliminated from all surfaces, foundations, and grammar. All data construction through cons and the list library function.
- **Anonymous combiners forbidden in all surfaces.** Every gamma or lambda must be named via define. This constraint — motivated by spacy’s inability to delimit inline anonymous combiners — is now universal to maintain surface equivalence.
- **Two-tier vocabulary: foundations and base library.** The vocabulary is organized by a single criterion: can it be written in Möbius? Foundations (~34) require the evaluator or runtime. Base library combiners are Möbius programs with content hashes. The terms “primitive”, “sugar”, and “registration-time form” are retired.
- **Foundations are not irreducible.** Some foundations have equivalent expansions in terms of others (lambda in terms of gamma, and in terms of if). These are semantic equivalences the compiler may exploit, not a hierarchy.
- **Capsule types use non-generative identifiers.** encapsulation-type takes an integer smaller than 2^{12} . Same identifier anywhere = same type. Library combiners capsule-constructor, capsule-predicate, capsule-unwrap simplify the common pattern.
- **Type predicates are foundations.** integer?, float?, char?, string?, pair?, box?, combiner?, continuation? are explicit foundations.
- **display is a foundation** (requires OS interaction). equal?, not, continuation-extend are base library.
- **Truthiness clarified.** Only #false is false. 0, #nil, "", #void are all true. or returns first non-#false value.
- **#void and #eof are pre-defined singletons.**
- **Hex integer literals.** 0xC0FF33 is valid reader syntax for positive integers.
- **Comment syntax per surface.** Round: ;. Curly: //. Spacy: #. All surfaces support #; datum comments.
- **Mutual recursion moved to open questions.** Top-level mutually recursive definitions are not yet supported. The design of definition bundling is deferred.
- **error added to base library.** Takes exit code, message, and a tree.
- **Interpreter milestones section removed** (implementation details, not specification).

This document records the design of Möbius as understood in April 2026. It is a working manual, not a final standard. The “0” in R RM reflects this: it is the revision before revision, the seed before the tree.

Same hash, different names. Same kaleidoscope, different colors. The mobius-store is

append-only. The hashes remain. No more ghosts.